

AD Domain Migration and Infrastructure Realignment

Project Report

by Rob Das

DTEK Consulting Services Ltd.

Alberta, Canada

January 2025 – May 2026

This project report documents the Active Directory domain migration and infrastructure realignment project. The target audience is: systems administrators.

1. Executive Summary

This project was initiated in response to an Active Directory domain-level disruption that triggered a formal review of the underlying infrastructure. The disruption exposed the potential single point of failure posed by the original primary domain controller, a Windows Server 2012 domain controller for the company's Active Directory domain. An attempted failover to the secondary domain controller revealed that it was non-functional, likely due to extensive legacy data corruption and aging replication metadata.

Given the instability and age of the domain environment, a decision was made to transition away from the original domain entirely, which led to the creation of a new, cleanly deployed Active Directory domain built using best practices and updated Windows Server systems. The project entailed building new domain controllers, rearchitecting infrastructure roles, migrating workstations and services, centralizing windows updates, upgrading from Windows 10 to 11, and establishing a repeatable, standards-based configuration for long-term stability. The project followed a low-code approach to simplify long term maintenance and support.

The result is a more reliable, secure, and maintainable domain environment that removes legacy dependencies and lays a foundation for future scalability.

TABLE OF CONTENTS

1. Executive Summary	1
2. Project Overview	3
3. Project Outcomes.....	4
4. Infrastructure Summary	6
5. Domain Architecture	9
6. System Configuration and Services	11
7. Migration Details	14
8. Migration of Interdependent Legacy Application	17
9. Implementing WSUS Patch Management.....	19
10. Windows 10 to Windows 11 Desktop Modernization	31
11. Linux as a First-Class Desktop Platform.....	41
12. Operational Standards	51
13. Testing and Validation	53
14. Outstanding Tasks and Future Plans	55
TROUBLESHOOTING GUIDE APPENDIX	57
A1 WSUS Troubleshooting	57
A2 Windows 11 Troubleshooting.....	61
A3 Linux Mint Troubleshooting.....	67
REFERENCES	72

2. Project Overview

The domain migration project began after a Windows Update maintenance cycle caused the original primary domain controller for the company's internal Active Directory domain, *dtek.internal*, to temporarily go offline. Despite the presence of a secondary domain controller, domain services did not fail over properly, resulting in a temporary domain-wide outage. Extensive troubleshooting revealed that replication between the controllers had long been broken, and the stale replication metadata within the domain's internal databases made recovery infeasible. The original secondary domain controller was demoted and removed from the domain, exposing the original domain controller as a potential single point of failure.

A temporary remediation effort was undertaken to stabilize the original primary domain controller so that core authentication services could continue functioning while a new environment was planned. During this period, attempts to promote a new secondary domain controller into the existing domain failed due to persistent data integrity issues.

At that point, the decision was made to build a brand-new domain, *d1.internal*, from the ground up, with clean Active Directory configuration, hardened DNS, and improved operational practices. A pair of domain controllers (*dc-01* and *dc-02*) were deployed on updated Windows Server systems hosted on dedicated Hyper-V hypervisors. Existing systems were assessed for migration, and a staged migration plan was developed to move production systems either into the new domain or into standalone WORKGROUP configurations, depending on their role.

The project's scope of work was scaled up to include a significant infrastructure realignment:

- Replace multi-purpose physical servers with role-dedicated hypervisors.
- Create a new virtualized file server.
- Migrate user profiles and application services.
- Establish operational standards and checklists for long-term maintainability and disaster recovery.
- Introduce centralized Windows Update management to streamline Windows patching and reduce admin costs.
- Upgrade all end-of-life Windows 10 machines with either Windows 11 or Linux Mint.

With the completion of the migration and decommissioning of the original primary domain controller, the company, DTEK Consulting Services Ltd., now operates on a more resilient and standards-compliant foundation.

3. Project Outcomes

3.1 IT Functionality Improved or Modernized

The domain migration project delivered multiple infrastructure and operational upgrades that increased manageability, security, and resilience:

- **Active Directory consolidation:** Legacy domain dtek.internal was fully retired, and all eligible systems now operate under the unified d1.internal domain with standardized Group Policy and account controls.
- **DNS structure and replication improved:** DNS zones were cleaned up, reverse lookup zones validated, and replication between domain controllers confirmed, improving name resolution reliability and administrative clarity.
- **Workstation standardization:** All domain-joined workstations now follow a consistent profile setup and permissions model. Manual transitions were verified and documented to simplify future onboarding.
- **Legacy systems preserved and stabilized:** A fragile, multi-tier application stack (involving Visual Studio, IIS, SQL Server, and TFS) was successfully migrated with no loss of functionality. Key dependencies were carefully re-established under the new domain.
- **Disaster recovery preparation introduced:** Clean Hyper-V exports of both domain controllers were created and archived, establishing a solid baseline for future DR planning.
- **Documentation culture reinforced:** Migration processes, DNS configuration, file share policies, and scheduled task management were all documented to a level suitable for junior administrators—building internal knowledge continuity.
- **Windows updates centralized and automated:** Monthly patches are now downloaded once to the WSUS server and distributed internally, reducing Internet bandwidth use and eliminating manual update checks on individual systems.
- **Windows 11 upgrades deployed across eligible machines:** All Windows 10 systems meeting minimum requirements were upgraded to Windows 11, improving security posture, boot performance, and compatibility with modern management tools. In-place upgrades were validated to preserve user profiles and application configurations, minimizing disruption.
- **Linux Mint adopted for legacy hardware:** Physical laptops with Windows 10 unable to support Windows 11 were transitioned to Linux Mint, extending device usability while reducing licensing costs. Core productivity functions were preserved, and domain integration was bypassed in favor of local account management.

3.2 Lessons Learned and Recommendations

This project revealed several insights and improvement opportunities for future infrastructure changes:

- **System interdependencies must be mapped early.** The tight coupling of legacy systems required significant effort to trace and preserve during the transition. Documenting app-to-service dependencies should become a standard pre-migration step.
- **Reverse lookup zones are often overlooked.** Their absence can lead to subtle network issues. We recommend including zone verification in the checklist for any future DC deployment.
- **Isolated Hyper-V hosts simplify DR but require intentional exports.** Keeping Hyper-V in WORKGROUP mode worked well operationally, but it places responsibility for exports squarely on local admins. Regular DR export schedules should be institutionalized.
- **Incremental wins matter.** While this project involved legacy systems, it provided opportunities to clean up stale DNS entries, update file permissions, and align naming conventions. Even legacy migrations can result in a more modern and maintainable environment when handled deliberately.
- **Keep junior administrators in mind.** Many of the operational checklists developed during this project can now serve as reusable training material—reducing onboarding friction and institutionalizing best practices.
- **Roll out WSUS in stages.** Starting with a single test system allowed safe evaluation before affecting production servers—minimizing risk while confirming real benefits.
- **System image restore is not a disk shrinking tool.** Using Windows Backup and Restore (Windows 7) proved valuable for creating fully functional duplicate VMs but cannot be relied on to reduce disk size.
- **Hardware limitations shaped OS deployment decisions.** Early compatibility checks identified systems unsuitable for Windows 11 upgrades, leading to Linux Mint replacements as an effective alternative.

4. Infrastructure Summary

This section provides a summary of the core infrastructure elements that were implemented, replaced, or restructured as part of the domain migration and infrastructure realignment project. It reflects the current state of the company's IT environment following the completion of the transition from dtek.internal to d1.internal and supporting infrastructure realignment.

4.1 Active Directory and Domain Controllers

The new d1.internal domain is served by two Windows Server 2019 virtual machine domain controllers hosted on separate Hyper-V hypervisors. These domain controllers were configured from scratch using Microsoft-recommended defaults, with replication configured and verified

- dc-01: Primary domain controller hosted on Hyper-V server sirius.
- dc-02: Secondary domain controller hosted on Hyper-V server titan.

4.2 Hyper-V Hosts

The company's virtualization infrastructure was reorganized to move away from multipurpose physical servers. All critical workloads were moved to virtual machines hosted on dedicated Hyper-V servers:

- lunar (Windows Server 2016, retained due to long-standing stability)
- europa (Windows Server 2019)
- titan (Windows Server 2019)
- sirius (Windows Server 2019)

Each hypervisor is configured as a standalone host in WORKGROUP mode and does not participate in domain services.

4.3 File Server Services

The file server infrastructure was reorganized to simplify the role of Hyper-V hosts and establish clear separation of responsibilities. Previously, europa served both as a Hyper-V host and as a file server. A new virtual machine named file-01, running Windows Server 2016, was created to assume all file server responsibilities previously handled by europa.

- file-01 is now the second file server in the organization.
- It is hosted on europa and uses a dedicated VHDX volume backed by local storage.

- File shares were migrated from europa's physical F: drive to file-01, and a new access model based on domain security groups was implemented to follow the principle of least privilege.

In parallel, significant cleanup efforts were made on an existing file server named lunar-file. While structural improvements were implemented, such as removing redundant shares and standardizing folder layout, additional work is still planned to bring its permissions model in line with the standards used on file-01.

4.4 Domain Membership

Each computer in the organization was reviewed for its function, and then reassigned to one of two *network-roles*:

- *domain member*: systems requiring Active Directory services were joined to d1.internal
- *workgroup system*: systems with no dependency on domain services remained or were moved to WORKGROUP

This structure ensures minimal reliance on domain services where unnecessary, reducing risk and simplifying future disaster recovery.

4.5 Legacy System Handling

Legacy systems to support dtek.internal were retired, demoted, or repurposed:

- the original primary domain controller was demoted and removed from Active Directory.
- the original secondary domain controller was decommissioned earlier in the project.
- Systems dependent on dtek.internal were reviewed and migrated case-by-case.

4.6 Database Servers

Two database servers are maintained within the updated infrastructure to support both production and development workloads:

- lunar-sql: A SQL Server 2014 system hosting databases used by multiple applications. It was migrated from the dtek.internal domain to d1.internal as part of the project.
- w16-cosmosdb: A Windows Server 2016 virtual machine that runs the Azure Cosmos DB Emulator for on-premises development and testing.

4.7 WSUS Server

A dedicated Windows Server Update Services (WSUS) server was deployed to centralize Windows update distribution within the d1.internal domain.

- WSUS2, a Windows Server 2016 powered WSUS server, serves as a local Windows update patch repository management system.
- WSUS reduces bandwidth usage and streamlines the monthly Windows update cycles by eliminating the need for systems to download Windows updates individually.

4.8 Tools and Utilities Used

The following tools were instrumental throughout the planning, execution, and validation phases of the project:

- Active Directory Domain Services (AD DS)
- DNS Manager
- Server Manager
- Hyper-V Manager
- PowerShell and Command Prompt
- Remote Desktop Connection (RDP)
- Visual Studio – for debugging web deployments and TFS integration
- IIS Management Console – for website configuration and bindings
- SQL Server Management Studio (SSMS) – for validating SQL connectivity and security
- Windows Server Update Services Console (WSUS) – to configure, sync, update approval
- Local Group Policy Editor (gpedit.msc) – for configuring WSUS client behavior
- ChatGPT – for architectural planning, PowerShell scripting, troubleshooting
- Linux – Terminal, Network Settings, File Manager (nemo), Backup (timeshift), RDP clients (remmina, xrdp), LibreOffice, Text Editor (xed), Sound/Video Device Manager

5. Domain Architecture

This section describes the Active Directory domain architecture.

5.1 Purpose and design of dc-01 and dc-02

The domain d1.internal is hosted by two domain controllers, both running Windows Server 2019 edition:

- dc-02 is hosted on titan
- dc-01 is hosted on sirius

All four Hyper-V hosts in the environment (lunar, europa, titan, sirius) operate independently and are treated as equal peers. Domain controller placement across different physical hosts was deliberately chosen to ensure domain availability in the event of hardware failure.

Both domain controllers provide:

- Active Directory Domain Services (AD DS)
- DNS Server
- Global Catalog

Checkpoints are explicitly avoided for both dc-01 and dc-02. On 2025-05-07, both domain controller VMs were shut down and exported to form a backup named:

```
d1-domain-controllers_2025-05-07.zip
```

5.2 Active Directory domain design and OU structure

The domain was created from scratch using the default Server Manager GUI workflow. No objects, configurations, or policies were imported from the legacy domain dtek.internal.

As of deployment, the directory contains only the default containers automatically created by Windows:

- Builtin
- Computers
- Domain Controllers
- ForeignSecurityPrincipals
- Managed Service Accounts
- Users
- *No Custom Organizational Units (OUs) have been created.*

5.3 DNS and DHCP configuration summary

DNS is AD-integrated and redundantly hosted on both domain controllers (dc-01 and dc-02).

- Primary zone: d1.internal
- Replication scope: To all domain controllers in the domain
- Reverse lookup zone for the 192.168.11.0/24 network was created as part of initial setup

DHCP is not managed by Windows Server. It continues to be provided by the network router at 192.168.11.1, which has not been modified and continues to serve leases to the entire subnet. This decision was made to avoid disrupting long-established network infrastructure.

5.4 Global catalog and FSMO role assignments

Global Catalog is a complete list of objects in the domain, and FSMO roles, or Flexible Single Master Operations roles, dictate which domain controller is responsible for specific types of updates and changes, preventing replication conflicts and ensuring the directory operates correctly. Both domain controllers (dc-01 and dc-02) are Global Catalog servers, as verified in Active Directory Sites and Services.

FSMO Role Holders are shown below (confirmed via “netdom query fsmo”):

- Schema Master: dc-01.d1.internal
- Domain Naming Master: dc-01.d1.internal
- PDC Emulator: dc-01.d1.internal
- RID Master: dc-01.d1.internal
- Infrastructure Master: dc-01.d1.internal

All FSMO roles are currently held by dc-01.

5.5 Group Policy overview

No custom Group Policies have been created or modified at this time. The following default GPOs exist:

- Default Domain Policy
- Default Domain Controllers Policy

These GPOs apply domain baseline settings. They have not been edited or extended.

6. System Configuration and Services

This chapter describes how the various systems are configured and the services they provide.

6.1 File Server Transition and Design Intent

Historically, europa, a voluptuous server blessed with a pair of ample hard drives, served dual roles as both a Hyper-V host and a file server. To simplify its responsibilities and improve manageability, file services were migrated to a new, dedicated file server virtual machine named file-01. File-01 was created specifically to offload user and departmental file shares from europa, allowing europa to operate solely as a Hyper-V hypervisor going forward. Europa was then chosen to host file-01, because of europa's ample hard drive capacity.

In parallel, substantial cleanup was performed on lunar-file, an older file server, to consolidate, remove redundant shares, and improve organizational clarity. Although more work remains—particularly in aligning lunar-file share permissions with the least-privilege model established on file-01—this effort marked a key step toward standardized and streamlined file service management.

Both file-01 and lunar-file are Windows Server systems joined to the d1.internal domain, managed via Server Manager and AD-integrated access controls.

6.2 Role Separation

This section discusses repurposing multi-role servers into dedicated hypervisors. Early in the environment's lifecycle, some systems were configured with multiple roles. Over time, these systems were repurposed to isolate hypervisor responsibilities from other server roles. Specifically:

- europa was reconfigured to remove file services and act solely as a Hyper-V host.
- lunar was converted from one edition of Windows Server 2016 to another to support Hyper-V features and licensing. It remains on 2016 for stability reasons.
- sirius and titan were built as Windows Server 2019 installations from the start, dedicated to Hyper-V hosting only.

This role separation ensures better isolation, performance, and ease of maintenance.

6.3 Local Admin vs Domain Access Control

Domain-joined servers and workstations now rely on domain-based user authentication via d1.internal. Where possible, local administrator access is restricted to domain-level admin

groups (D1\Administrators), rather than relying on per-machine local accounts. This approach simplifies credential management and improves auditing consistency.

Exceptions exist on standalone systems (such as those in WORKGROUP configurations) where local credentials are required. These systems are limited in scope and isolated by design.

6.4 Security Hardening: Least-Privilege

Security configurations follow a pragmatic model of high trust among internal staff. By default, all internal users are granted full access to shared file resources, consistent with company policy. File servers such as file-01 and lunar-file are configured so that most shares use “Everyone / Full Control” to streamline collaboration and minimize support overhead.

Exceptions are made for a small number of shares containing sensitive information—typically related to finance or system administration. These shares are explicitly secured using named domain accounts and group-based permissions to limit access.

Domain-based authentication is enforced across all servers and shares, ensuring only authenticated users can access internal resources. This model balances ease of access with reasonable protection of sensitive information.

6.5 Linux Mint Laptops

All laptops currently running Windows 10 that do not meet the minimum requirements for Windows 11 are being upgraded to Linux Mint. This change is driven primarily by the upcoming end-of-life for Windows 10 in October 2025, ensuring continued secure and supported operation without requiring hardware replacement. The primary users of these systems are developers and systems administrators, who rely on them for remote access to Windows-based infrastructure, file management, office productivity tasks, web browsing, and occasional local audio/video playback.

The migration approach focuses on replicating the essential capabilities previously available under Windows 10, using Linux Mint’s built-in features and user-friendly GUI tools. This includes configuring RDP access to Windows systems, enabling full multimedia support, setting up office document editing, and ensuring seamless integration into existing network and authentication environments. Select Linux tools that are similar to their Windows equivalents are installed and enabled to maintain operational continuity while minimizing retraining needs. The migration process ensures that users retain full access to internal systems, development environments, and administrative utilities, with minimal disruption to workflows.

6.6 Printing to TITAN\Canon

The Canon TS5300 printer is currently hosted on TITAN, a Windows Server 2019 machine operating in a standalone WORKGROUP configuration. TITAN is presently serving as the

company's network printer server, providing access to the Canon printer for both Windows 11 domain-joined workstations and Linux Mint laptops. This section outlines the configuration required to enable printing from both environments.

For Windows 11 machines joined to the D1 domain, the printer is accessed via the shared path \\TITAN\Canon. Initial setup must be performed under a local administrator account, such as GALAXY\Administrator, to install the Canon TS5300 driver and register the shared printer. Once installed, domain users can add the printer through standard Windows printer discovery or by manually specifying the network path. No additional driver installation is required for domain users after the initial setup.

Linux Mint laptops connect to the printer using the CUPS printing system with Samba support. Users should open the Printers utility, select "Windows Printer via Samba," and enter the path smb://TITAN/Canon. If prompted, credentials for TITAN must be supplied. The Canon TS5300 driver should be selected if available; otherwise, a generic Canon Inkjet driver may be used. A successful test page confirms connectivity and driver compatibility.

This configuration ensures that both Windows and Linux clients can print reliably to the shared Canon printer hosted on TITAN.

7. Migration Details

This section provides a detailed overview of the migration process, including the final placement of computers, rationale for domain membership decisions, and configuration of non-standard setups.

7.1 Final Placement of Computers

The table below summarizes the current placement of all known computers across the organization. Systems were either migrated to the d1.internal domain, placed in a WORKGROUP, or in special cases left unjoined due to technical constraints.

Domain / Workgroup	Role	Number of Computers
d1.internal	App Server	3
	Batch Jobs on Task Scheduler	1
	Database Server (SQL-Server)	1
	Developer Workstation	1
	Documentation Workstation	1
	File Server	1
	General Purpose Workstation	3
	Primary DC	1
	Secondary DC	1
	Web Hosting - Dev/Test	1
	Web Hosting - Production	1
WORKGROUP	Database Server (Cosmos DB)	1

Domain / Workgroup	Role	Number of Computers
	Decommissioned DC	1
	Developer Workstation	2
	General Purpose Server	3
	General Purpose Workstation	6
	Hyper-V Host	4
mylab.local	Laboratory Computer	1
n/a	Linux Computers	3

7.2 Rationale for Domain Membership Decisions

Most systems performing core infrastructure functions (e.g., file sharing, application hosting, SQL Server) were joined to the d1.internal Active Directory domain for centralized identity management and policy enforcement.

However, several systems were intentionally not joined to the domain due to the following factors:

- Hyper-V Hosts (sirius, titan, lunar, europa) were excluded from domain membership to avoid dependency on AD availability and to improve isolation for disaster recovery purposes.
- Unsupported Operating Systems such as Windows 10/11 Home Edition and Linux do not support AD domain joining.
- Application Incompatibility: The CosmosDB emulator on w16-cosmosdb is known to behave unpredictably in domain environments, and was deliberately left unjoined.
- Unassigned / Spare Systems: Some computers have not yet been assigned for production use, so domain decisions have been deferred.

7.3 Configuration of Non-Standard Setups

Some systems deviate from standard deployment due to historical decisions or unique workloads:

- lunar-sql and lunar-tfs retain legacy TFS + SQL Server roles with domain-linked authentication still being cleaned up.
- The web servers (eu-web, webserv-test) required careful DNS and authentication coordination with SQL-based services on lunar-sql using Windows-integrated credentials.
- A few legacy computers had dual identities (e.g., previously joined to dtek.internal, then rejoined to d1.internal) which occasionally caused duplicate SPNs or SID issues during migration. These were manually resolved.

8. Migration of Interdependent Legacy Application

One of the most intricate and time-consuming parts of the domain migration and infrastructure alignment project was the coordinated transfer of five legacy systems that collectively supported a multi-tier, front-end web application and its supporting infrastructure. These systems had to be migrated together due to tight operational coupling, shared authentication requirements, and historical design constraints.

8.1 Systems Involved

The following systems were involved:

- devbox-1: Developer workstation for legacy Visual Studio projects
- eu-web: Production front-end web server (MVC application)
- webserv-test: Staging/test web server (mirrors production setup)
- lunar-sql: Database server hosting Webserv-dev, Webserv-test, and Webserv-prod (SQL Server 2014)
- lunar-tfs: Source control server running TFS (Team Foundation Server) with its own local SQL Server instance

8.2 Domain Authentication Dependencies

The application stack relies on Windows-integrated authentication for both web-server-to-database communication and user-level access controls. This meant the migration from dtek.internal to d1.internal could not break or disrupt:

- Application pool identities used in IIS
- SQL Server logins mapped to Active Directory domain accounts
- Security roles in individual SQL databases
- External access via leased domain name and its static IP

All interdependencies had to be preserved or recreated during the transition to d1.internal, which required:

- Adding new SQL Server logins for the domain-joined machine accounts: D1\EU-WEB\$, D1\WEBSERV-TEST\$, and D1\DEVBOX-1\$
- Ensuring corresponding roles and permissions were reassigned to these new accounts in the Webserv-dev, Webserv-test, and Webserv-prod databases
- Reconfiguring IIS application pools and web.config files

- Carefully staged system shutdowns and reboots to coordinate AD changes

8.3 TFS Source Control Migration

The lunar-tfs server hosted a legacy TFS environment backed by its own SQL Server instance. The migration involved:

- Domain join to d1.internal without breaking TFS service identity bindings
- Verifying that TFS could still access its databases and authenticate users
- Confirming continued access to project history, check-ins, and team settings

8.4 Testing and Validation

Significant testing was conducted to confirm:

- Database connectivity and application function post-migration
- Web application online availability through leased public domain name
- Correct mapping of SQL logins to domain accounts
- TFS service availability and user access

This combined migration was one of the most delicate operations in the project due to the number of moving parts, the use of Windows authentication across multiple tiers, and the live production nature of eu-web.

9. Implementing WSUS Patch Management

Windows Server Update Services (WSUS), a software update management system, allows systems administrators to manage deployment of Microsoft product updates to client computers on a network.

9.1 Purpose and Overview

To modernize monthly patching and reduce overhead, a centralized update system was deployed using Windows Server Update Services (WSUS). Updates are now downloaded once and distributed internally, cutting bandwidth use, standardizing patch levels, and improving control over deployments.

9.2 Current WSUS Automation Schedule

In order to deploy updates in an automated fashion, we have adopted the following schedule.

Event	Time (MST)	Purpose
Microsoft Patch Tuesday	Second Tuesday each month at 11:00 AM	Monthly release of updates by Microsoft
WSUS Daily Sync	Daily at 9:00 AM	Pulls updates from Microsoft
Admin Approval Window	2:00-4:00 PM on the day after Patch Tuesday	Manual review and approval of updates
Daily Client Scan, triggered via PowerShell script <i>S:\Code\forcewuscan.ps1</i>	Daily at 6:00 PM Exception: WSUS server daily at 5:00 PM	Initiates update detection on clients
Update Installation, triggered via Policy	Daily at 1:00 AM Exception: WSUS server daily at 9:00 PM	Applies approved updates
Daily Client Restart via PowerShell script <i>S:\Code\forcewurestart.ps1</i>	Daily at 6:00 AM Exception: WSUS server daily at 10:00 PM	Forces restart to complete update installation
Weekly unconditional restart via PowerShell script <i>S:\Code\forceunconditionalrestart.ps1</i>	Weekly, Sundays at 12:00 PM On Windows 11 computers only	Clears temporary update failures
Monthly database cleanup <i>S:\Code\wsusmaint.ps1</i>	The first of every month at 3:00 AM	Monthly maintenance script responsible for keeping the WSUS environment responsive and efficient.

This schedule has proven reliable and balances timely patching with minimal disruption.

Daily items are independent of each other. If ran in the wrong order (perhaps due to an outage) they will automatically catch up the following day, once the correct order has been re-established.

However, even with this level of automation, Windows Updates can still be compromised by unexpected shutdown in the middle of an update (possibly due to power outages). To resolve this, see the troubleshooting guide at the end of this chapter.

Hyper-V hypervisor hosts are registered with Active Directory and WSUS and follow the standard WSUS automation schedule. Windows Update operations on a Hyper-V host are independent of update activity within guest virtual machines. A host reboot results in a controlled power cycle of guest VMs and does not corrupt guest operating systems, virtual disks, or in-progress updates. Since updates and restarts occur outside business hours and no clustered workloads are in use, Hyper-V hosts can safely follow the same WSUS standards and procedures as other Windows servers, with the WSUS server itself remaining the only exception.

Since Windows Server 2012 R2 reached end of extended support in 2023 and is no longer eligible for security or quality rollups, these systems are excluded from the WSUS automation framework. However, Microsoft continues to release the monthly Windows Malicious Software Removal Tool (MSRT) for 2012 R2. To ensure these updates are applied with minimal overhead, these servers are configured through the Control Panel → Windows Update settings to “Install updates automatically”.

9.3 WSUS Server Infrastructure

The WSUS server wsus2.d1.internal, deployed for centralized update management, runs Windows Server 2016 Standard, is joined to d1.internal, and is hosted on the Sirius Hyper-V system. It was installed using Server Manager and is hosted under IIS. No SSL certificate has been configured at this stage, so the system currently uses HTTP for client communication.

Key configuration details:

- Hostname: wsus2.d1.internal
- OS: Windows Server 2016 Standard
- Domain: d1.internal
- Virtualization: Hosted on Sirius (Hyper-V)
- Database: Windows Internal Database (WID)
- Update Storage: E:\WSUS
- IIS Ports: 8530 (HTTP), 8531 (HTTPS)
- Targeting Mode: Server-Side Targeting

This WSUS deployment uses *server-side targeting*, where all clients appear in the Unassigned Computers group and are manually moved to WSUS groups (such as Win2016) via the WSUS console. Server-side targeting is the default WSUS configuration, found under:

```
WSUS Console > Options > Computers > Use the Update Services console
```

If desired, client-side targeting can be enabled instead by switching the above option and applying the "Enable client-side targeting" GPO setting on clients.

The system was designed with recovery and maintenance in mind. Because it uses WID and stores all updates locally, the virtual machine can be exported and restored easily using Hyper-V Manager. No external database or file share dependencies exist, making the configuration highly portable.

9.4 Non-Default Configuration

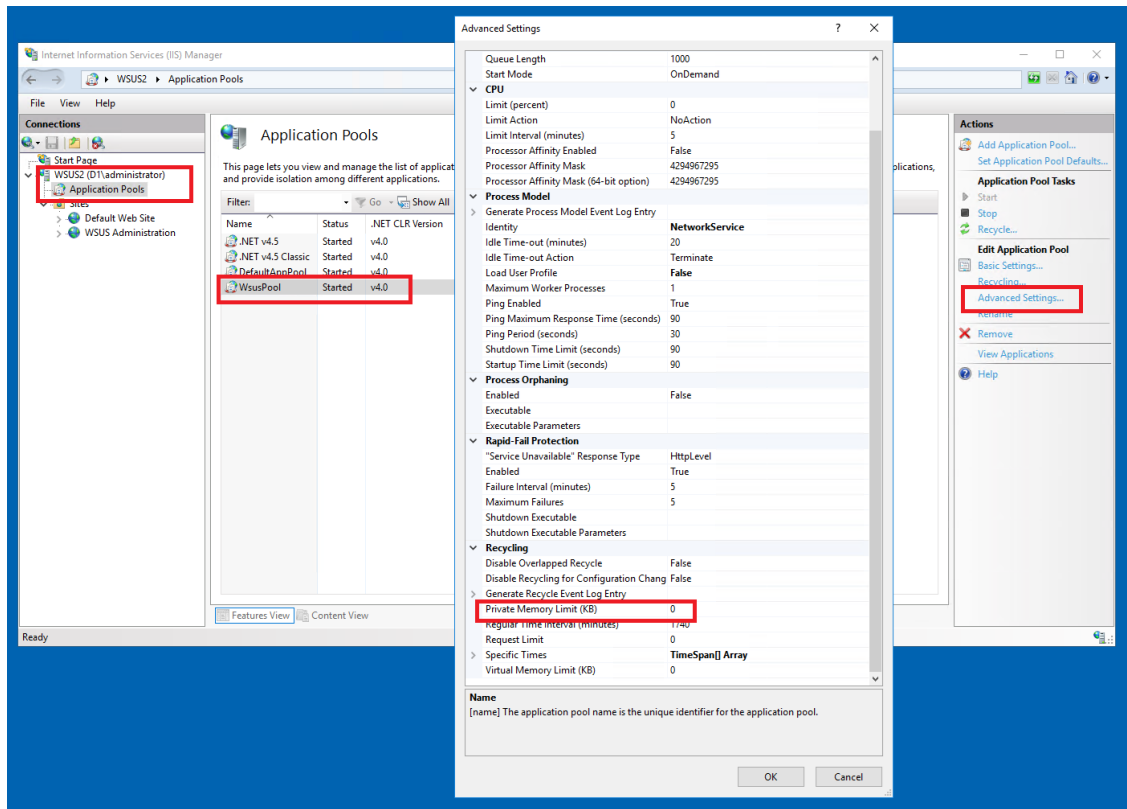
To optimize WSUS for its virtualized environment and ensure scalability, several settings were adjusted from their defaults.

Hyper-V VM Settings:

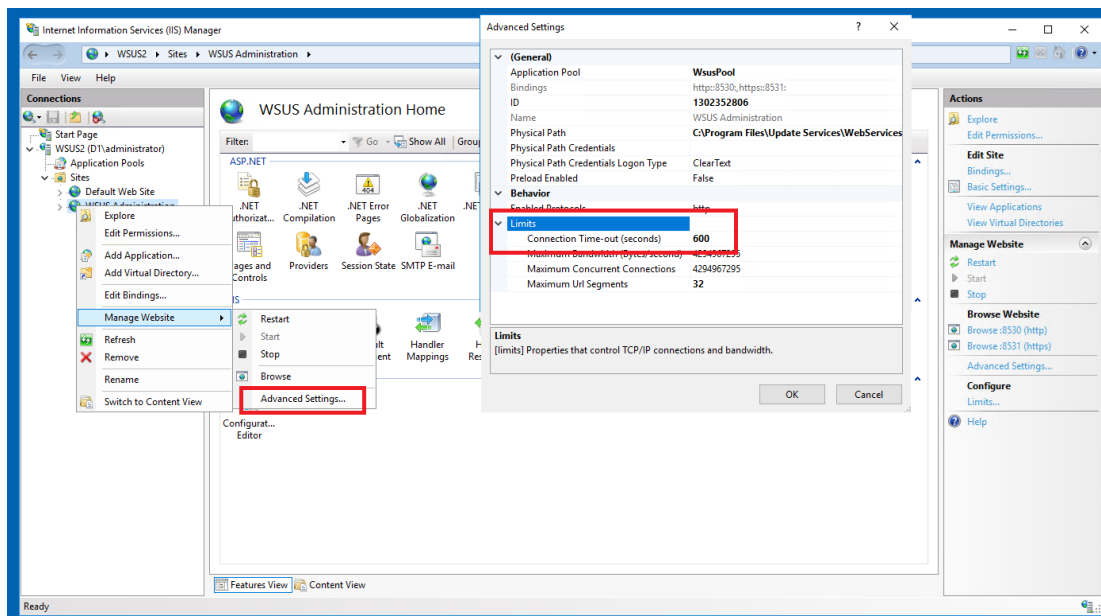
- Dynamic Memory: 2048–8192 MB
- Processor: 1 virtual CPU

This balances performance with the resource constraints of the Sirius host.

IIS Configuration:



- WsusPool Private Memory Limit: 0 (unlimited)



- Connection Timeout: 600 seconds

After adjusting, restart the WSUS IIS site: right-click WSUS Administration → Manage Website → Restart, or run iisreset from an elevated command prompt.

These prevent console hangs and allow longer-running operations during sync and approval.

WSUS Targeting and Grouping:

- Server-side targeting is enabled (clients assigned manually via WSUS console).
- Groups are organized by OS version, beginning with Windows Server 2016.

This supports phased rollouts and OS-specific update approvals.

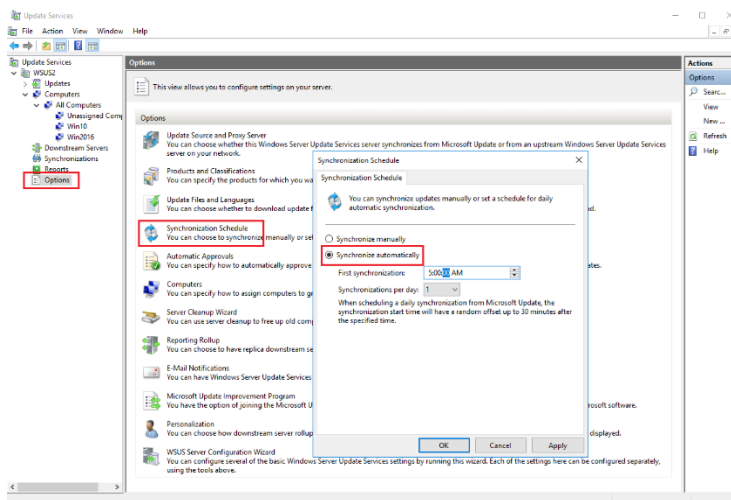
Enabling WSUS Synchronization Schedule

Open

WSUS Console → Options → Synchronization Schedule

Select

- Synchronize automatically
- Set to run daily at a fixed time



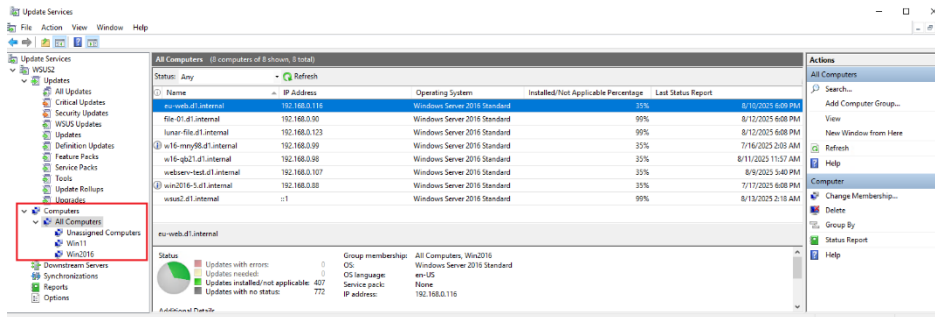
WSUS will automatically fetch new updates from Microsoft every morning.

9.5 Steps for Adding Additional Clients

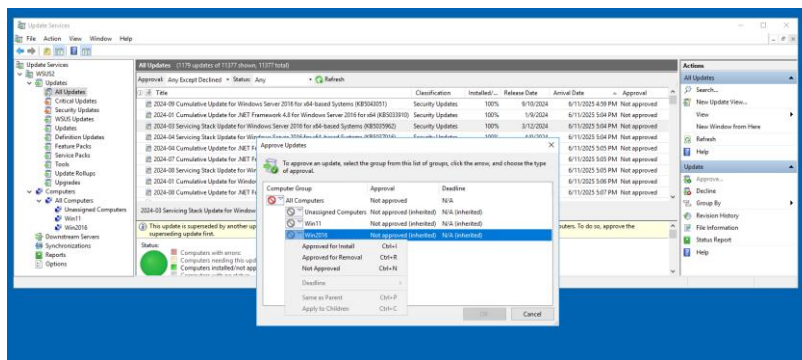
Clients are organized by OS on the WSUS server using WSUS computer groups.

Server Steps

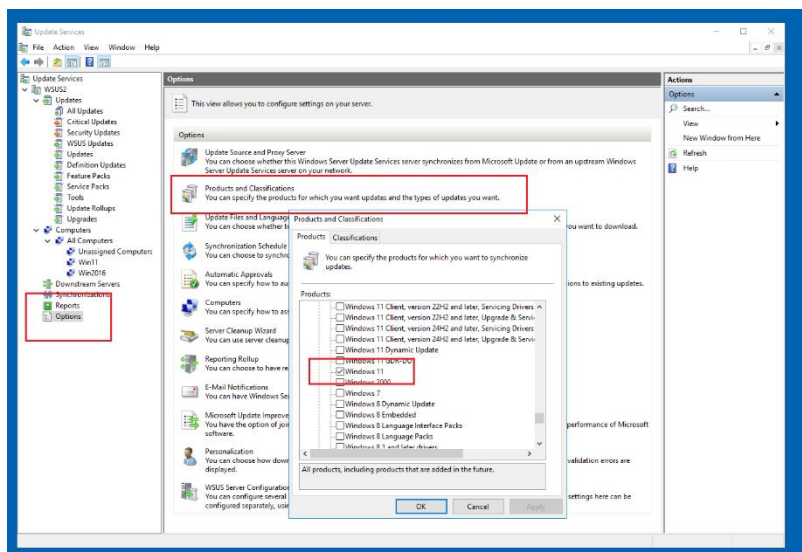
Currently we have separate computer groups, Win11 and Win2016, for Windows 11 updates, and Windows Server 2016 updates.



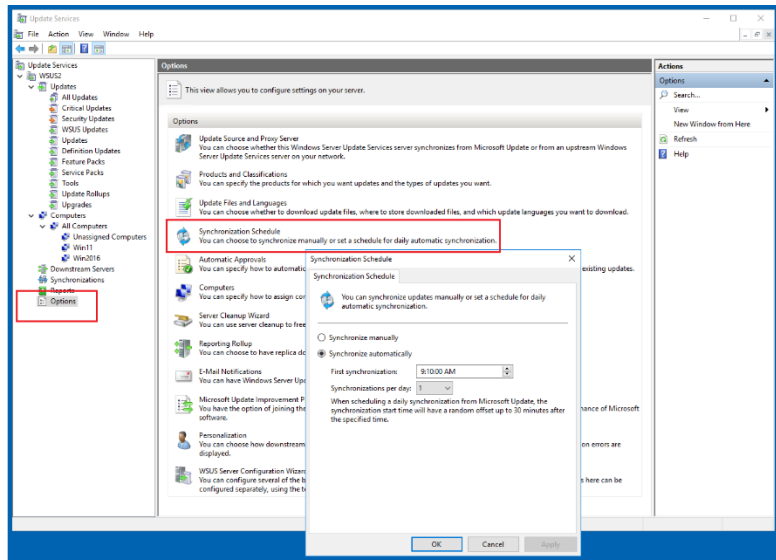
When a client first contacts the server, it is automatically added to the Unassigned Computers group, and must be manually moved into one of the computer groups so that it receives the correct Windows updates. The Windows updates must be approved each month and targeted to the correct computer group. This is done manually by the sys admin the after they are published by Microsoft on “Patch Tuesdays”.



When adding new computer groups such as Win11, you need to select the products that determine the updates for the new computer group. This is done in the WSUS console > Options > Products and Classifications.



This will tell WSUS what to download during synchronization. Synchronization is when the WSUS server reaches out to Microsoft's online update service and fetches the updates corresponding to the products that were selected. Synchronizations are scheduled from the WSUS console > Options > Synchronization Schedule



Client Steps

To onboard new clients into the WSUS infrastructure, follow these steps to ensure proper registration and update delivery. This process ensures correct WSUS registration and fully automated update delivery using a dual mechanism: Local Group Policy + Scheduled script. "Server-side targeting" is used and WSUS-related Group Policy is applied locally on each client.

Step 1 – Join the Domain

Ensure the client system is joined to the d1.internal Active Directory domain.

Step 2 – Apply Local Group Policy

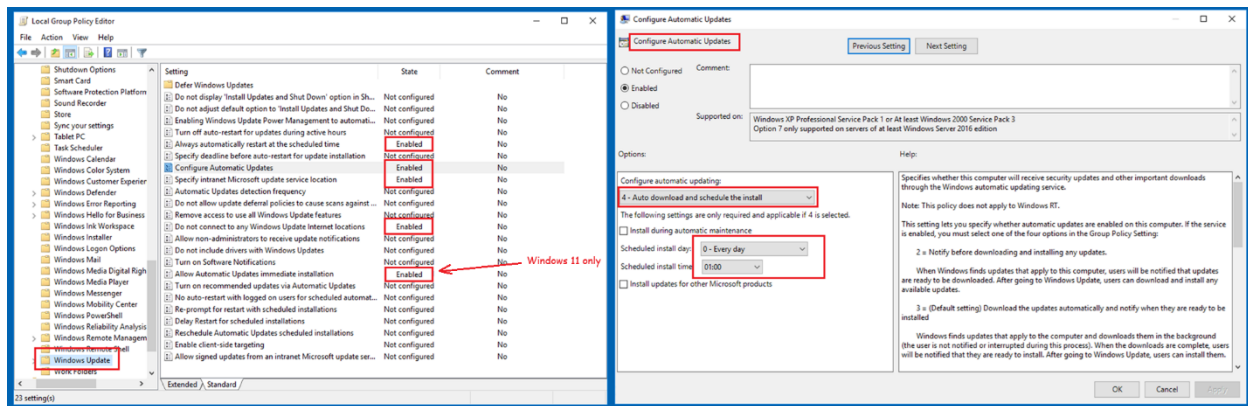
On the client, open the Local Group Policy Editor (gpedit.msc)

Local Computer Policy > Computer Configuration > Administrative Templates >
Windows Components > Windows Updates

and set these policies:

1. Enable "Specify intranet Microsoft update service location"
 - a. Detection server: http://wsus2.d1.internal:8530
 - b. Statistics server: http://wsus2.d1.internal:8530
 - c. Leave alternate download server not configured.
2. Enable "Configure Automatic Updates"

- a. "4 - Auto download and schedule the install"
 - b. Scheduled install day: "daily"
 - c. Scheduled install time: "1:00 AM"
3. Enable "Do not connect to any Windows Update Internet locations"
this prevents fallback to Microsoft online
4. Enable "Always automatically restart at the scheduled time". This enforces a restart 15 minutes after installation completes, even if users are logged in.
5. **For Windows 11 systems only:**
Enable "Allow Automatic Updates Immediate Installation". This is needed to ensure all eligible updates install automatically without requiring user interaction, to align Windows 11 with the automated update behaviour used by Windows Server.



Apply the policies using:

```
gpupdate /force
```

Important: We use server-side targeting; do not configure "Enable client-side targeting".

Step 3 – Trigger WSUS Registration

On the client, go to Settings > Windows Update and click Check for updates. This will initiate contact with wsus2 and register the client in WSUS under Unassigned Computers.

Wait for the client to appear.

Note: A client in "Unassigned Computers" will not receive approved updates unless explicitly moved to a targeted group.

Step 4 - Automate Update Detection via Scheduled Script

Using Windows Task Scheduler on the client computer, create a scheduled task that runs a designated PowerShell script according to the WSUS automation schedule shown previously in this chapter. The script is on the network at this path:

```
\\lunar-file\share\Code\forcewuscan.ps1
```

Configure the task as follows.

- Task name: Force Windows Update Scan
- Program/script: powershell.exe
- Arguments:

```
-ExecutionPolicy Bypass -File "\\lunar-file\share\Code\forcewuscan.ps1"
```

This initiates the monthly “patch Tuesday” update process, which sometimes gets ignored by the overnight “Configure Automatic Updates” policy.

Alternatively, you can check to see if there is a task scheduler import script named \\lunar-file\share\Code\Force Windows Update Scan.xml

Step 5 - Automate the Restart Every Morning

On some client computers, the “Always automatically restart at the scheduled time” policy does not reliably enforce restarts after update installation. To ensure updates are completed, create a scheduled task in Windows Task Scheduler to force a system restart according to the WSUS automation schedule shown previously in this chapter.

Configure the task as follows.

- Task name: Force Windows Update Restart
- Program/script: powershell.exe
- Arguments:

```
-ExecutionPolicy Bypass -File "\\lunar-file\share\Code\forcewurestart.ps1"
```

This guarantees that computers finish installing updates if a Windows Update reboot is pending, without requiring user interaction.

Alternatively, you can check to see if there is a task scheduler import script named \\lunar-file\share\Code\Force Windows Update Restart.xml

Step 6 – Add a Weekly Unconditional Restart (Windows 11 Only)

Windows 11 desktops occasionally enter a temporary update failure state that does not automatically clear until the next cumulative update is released. To ensure these systems

remain healthy and continue to receive updates without manual intervention, create a scheduled task in Windows Task Scheduler to perform a weekly unconditional restart.

Configure the task as follows.

- Task name: Force Unconditional Restart
- Program/script: powershell.exe
- Arguments:

```
-ExecutionPolicy Bypass -File "\\lunar-file\share\Code\unconditionalrestart.ps1"
```

This scheduled restart ensures that Windows 11 clients clear transient update issues and remain fully compatible with the WSUS automation workflow.

Alternatively, you can check to see if there is a task scheduler import script named \\lunar-file\share\Code\Force Unconditional Restart.xml

Step 7 – Assign to WSUS Computer Group

In the WSUS console, move the new client to the appropriate group (e.g., Windows Server 2016) to control which updates it receives.

Step 8 – Monitor Update Status

To confirm update detection and installation:

- Check the client's status in the WSUS console.
- Check the script log file \\lunar-file\share\Code\forcewu-log.txt
- Check Update History on the client to verify that approved updates are applied
- On the client, inspect: C:\Windows\SoftwareDistribution\ReportingEvents.log for scan/install logs
- On the client, run `Get-WindowsUpdateLog`

This creates a readable WindowsUpdate.log file on the desktop. Confirm that update sources reference wsus2.d1.internal.

Note: If the client is already fully patched and you wish to validate WSUS-based update delivery, you may uninstall the most recent monthly cumulative update then restart and re-scan. This forces WSUS to reoffer the update, allowing for end-to-end testing.

9.6 WSUS Operational Workflow

This section outlines the standard operational process for approving and deploying updates using WSUS. These procedures form the baseline for ongoing patch management in our environment. We have standardized on the Server-Side Targeting approach, not the Client-Side

Targeting approach, for simplicity. And we have adopted conventions based around our Current WSUS Automation Schedule, shown earlier in this chapter.

WSUS Computer Groups

Client systems are organized in WSUS using server-side targeting, with each supported operating system assigned to its own WSUS computer group. For example, Win2016 for Windows Server 2016 systems.

This structure allows updates to be approved either for specific operating systems or for all computers, depending on the nature of the update.

Update Approval Process

Update approvals are performed using the WSUS console. Updates can be approved for individual computer groups (e.g., Win2016), or all Computers group (for critical or universally applicable updates)

Updates are not auto-approved. Each month's updates (typically released on Patch Tuesday) are reviewed and approved by a sys admin.

Client Update Retrieval

Once an update has been approved for the appropriate group, client systems will detect and download the update automatically based on their local Group Policy settings and Windows Update's internal schedule. In normal circumstances, this process requires no manual intervention.

All WSUS settings are applied via local Group Policy, not domain-level GPOs. This provides decoupling from domain controllers and simplifies troubleshooting.

Manual Overrides (if needed)

In rare cases where a client does not appear to be updating or checking in:

- A manual sync can be triggered using `wuaclt /detectnow` and `wuaclt /reportnow`
- Review the client's WindowsUpdate.log using `Get-WindowsUpdateLog`
- Check client is in the correct WSUS group and the update was approved for that group

Server Synchronization

Participating WSUS systems are configured for automatic daily synchronization.

Client Polling Interval

The WSUS clients automatically poll the server for updates according to the schedule published above in the section entitled Current WSUS Automation Schedule.

Monthly Database Maintenance

wsusmaint.ps1 is the monthly WSUS maintenance script responsible for keeping the Windows Server Update Services environment responsive and efficient. It connects to the Windows Internal Database (WID) and performs a full index rebuild on SUSDB, the “Software Update Services Database” that stores update metadata, approvals, computer status, and synchronization history. After reindexing, the script runs wsusutil reset, which forces WSUS to reconcile its metadata with the contents of the WSUSContent directory—essentially verifying that every update file referenced in the database exists on disk and removing references to any that do not. This process helps maintain consistency between the database and the update files stored on the server.

10. Windows 10 to Windows 11 Desktop Modernization

10.1 Overview

The transition from Windows 10 to Windows 11 became a strategic priority as Microsoft announced the end of support for Windows 10 in October 2025. This chapter documents the evaluation, planning, and execution of the Windows 11 replacement initiative across all eligible corporate systems. The goal was to ensure continued security, compliance, and operational stability while minimizing disruption and preserving existing workflows.

10.2 System Categorization

All Windows 10 systems were grouped into three categories based on their usage patterns and operational roles. This categorization ensured that each system received an appropriate modernization path aligned with its purpose and future value to the organization.

10.2.1 Dormant Virtual Machines

The first category consisted of dormant virtual machines. Although these systems were not actively used, they represented a valuable pool of pre-configured workstations that could be rapidly provisioned as needed. Each dormant VM was upgraded to Windows 11 to take advantage of Microsoft's free licensing entitlement and to maintain a ready-to-deploy inventory of modernized systems.

10.2.2 In-Use Virtual Machines

In-use virtual machines, that were actively assigned to administrative, development, or operational tasks, required continuity of service and minimal disruption. The upgrade strategy in this case prioritized stability, minimal downtime, and preservation of user environments. These systems were fully upgraded to Windows 11 using the standardized procedures described later in this chapter. This approach preserved existing workflows while bringing the systems into a supported and secure state.

10.2.3 Physical Machines

Physical laptops posed a different challenge due to their hardware limitations and licensing restrictions. In these cases, systems were upgraded by replacing the original Windows 10 installation entirely with Linux as described in the next chapter. This approach ensured that all physical systems—regardless of age—remained secure, functional, and aligned with the company's operational needs.

10.3 Upgrade Readiness Checks

Before upgrading any Windows 10 system to Windows 11, a small set of readiness checks must be completed to ensure the installation is healthy, properly licensed, and capable of accepting the upgrade. These checks apply equally to active systems, dormant virtual machines, and any future rebuilds that may be required.

1. **Verify system health** — confirm the system boots normally and shows no outstanding servicing or repair issues.
2. **Confirm activation status** — ensure Windows 10 is properly licensed and activated.
3. **Apply all available Windows 10 updates** — update the system until no further updates are offered.

These readiness steps establish a consistent baseline for all systems and prevent upgrade failures caused by incomplete servicing, licensing issues, or outdated components.

10.4 Upgrade Requirements and Methods

Upgrading Windows 10 virtual machines to Windows 11 requires meeting specific system prerequisites and selecting an appropriate upgrade method. This section outlines those requirements and describes the supported paths used to complete the upgrade.

10.4.1 Windows 11 System Prerequisites

Each virtual machine must meet Windows 11 system requirements before an upgrade can be attempted.

Rebuilding a Windows 10 virtual machine is only appropriate when the system is dormant, unassigned, or being repurposed, allowing a clean baseline to be created before upgrading to Windows 11. It should not be used for active development or production VMs where existing toolchains and configurations must be preserved.

The minimum system requirements for a virtual machine to upgrade to Windows 11 are

- 64-bit processor architecture
- Two or more CPU cores available to the virtual machine
- Minimum 4 GB RAM assigned to the virtual machine
- Minimum 64 GB system disk capacity
- UEFI firmware (Generation 2 virtual machine)
- Secure Boot enabled
- TPM 2.0 enabled for the virtual machine

These requirements are checked by the Windows 11 Setup. If any check fails, the upgrade will not proceed.

To improve upgrade reliability and reduce post-upgrade issues, systems should also be fully updated and verified as healthy prior to beginning the upgrade, and the C: drive should have 20 to 25 GB of free space available for the upgrade process.

To prepare a Windows 10 virtual machine to meet the Windows 11 prerequisites, the following actions are to be performed.

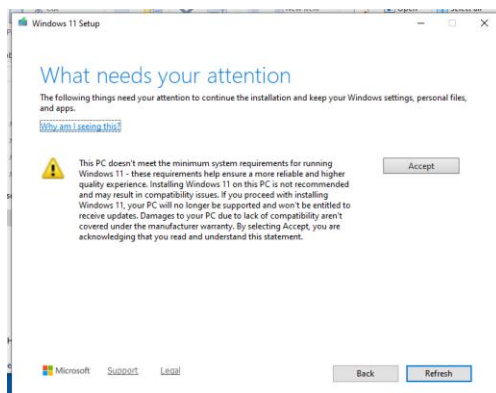
- Verify the virtual machine is configured with a 64-bit Windows 10 installation
- Increase the number of virtual CPU cores to two or more, if required
- Adjust assigned memory to at least 4 GB
- Ensure the system disk provides at least 64 GB capacity, including cleanup or expansion if necessary
- Confirm the virtual machine is configured as Generation 2 (UEFI)
- Enable Secure Boot in the virtual machine firmware settings
- Enable TPM 2.0 for the virtual machine using virtual TPM support

Because none of the Hyper-V hosts use CPUs listed as officially supported for Windows 11, a registry policy is required to allow the upgrade to proceed.

Add the following registry key if it's not already present.

- Key: HKEY_LOCAL_MACHINE\SYSTEM\Setup\MoSetup
- Value name: AllowUpgradesWithUnsupportedTPMOrCPU
- Type: REG_DWORD
- Value: 1

Note: When using the AllowUpgradesWithUnsupportedTPMOrCPU registry bypass, you may see this warning.



Accepting this warning allows the upgrade to proceed, with the understanding that the system will operate outside of Microsoft's supported configuration.

Reducing the size of the Windows 10 system disk before upgrading to Windows 11 is a recommended preparatory step for long-lived virtual machines. While this was not consistently completed during the initial rollout, it remains relevant for future rebuilds or repeat upgrades. The detailed procedures for shrinking and compacting VHDX files are documented later in this chapter.

10.4.2 Supported Upgrade Methods

Windows 11 can be installed using the Installation Assistant or an ISO file. The Installation Assistant requires internet access and enforces strict hardware checks. Using an ISO allows manual setup, offline upgrades, and easier bypass of hardware requirements.

The ISO-based upgrade was selected because it provides a predictable, repeatable process across all virtual machines. It does not require internet-based validation during setup, and works consistently even on hypervisors with unsupported CPUs. It allows reuse of a single ISO across multiple systems. It also allows upgrades to be performed offline, which is useful when preparing multiple VMs or working in controlled network environments.

10.4.3 Hyper-V Upgrade Paths

Different upgrade paths are required because virtual machines may be in varying states of use, readiness, and licensing.

10.4.3.1 In-Place Upgrade of an Existing Windows 10 VM

An in-place upgrade is used when an existing Windows 10 virtual machine must retain its current configuration and workload.

1. Start the Windows 11 Setup

Start the Windows 10 VM with the Windows 11 installation ISO mounted as the virtual DVD drive, but don't boot from the DVD drive, boot as normal from the C: drive. Once Windows 10 is running, open File Explorer, browse to the mounted ISO, and run *setup.exe* as an administrator.

2. Proceed through the installer

Accept the license terms, choose Keep personal files and apps, and continue through the prompts until the upgrade begins.

3. Allow the upgrade to complete

The upgrade typically takes 30–60 minutes and includes several reboots. After the final restart, Windows Update may run in the background for several minutes before the desktop appears.

The in-place upgrade preserves all existing user accounts, applications, and data, and it does not require the virtual machine to be joined to a domain.

10.4.3.2 New Windows 11 VM From Windows 10 License

A fresh upgrade path is used when there is no existing Windows 10 virtual machine to upgrade in place, but a valid Windows 10 license is available for activation. This applies to any transferable or unused Windows 10 license, including retail keys, subscription entitlements, or licenses reclaimed from decommissioned hardware.

1. Install Windows 10 in a new virtual machine

Create a new VM that meets the Windows 11 prerequisites described earlier in this chapter. Install Windows 10 and activate it using the available Windows 10 license key.

2. Run the Windows 11 Setup from inside the VM

Start the Windows 10 VM with the Windows 11 ISO mounted as the virtual DVD drive, but allow it to boot normally from the C: drive. Once Windows 10 is running, open File Explorer, browse to the mounted ISO, and run setup.exe as an administrator. Choose Keep personal files and apps and continue through the installer until the upgrade begins.

3. Allow the upgrade to complete

The upgrade typically takes 30–60 minutes and includes several reboots. After the final restart, Windows Update may run in the background for several minutes before the desktop appears.

This method is used when no Windows 10 VM exists to upgrade in place. It provides a clean Windows 11 environment activated through a valid Windows 10 license, regardless of how that license was originally obtained.

10.4.4 Post upgrade tasks

Detach the ISO from the virtual DVD drive and confirm the VM is running Windows 11 using *winver*. The registry bypass key may be removed, though leaving it in place causes no issues.

10.5 Moving Windows 11 VMs Between Hypervisors

Windows 11 virtual machines were moved between Hyper-V hosts running Windows Server 2016 or 2019, but the required steps varied depending on Windows 11 hardware requirements. Here are the three migration patterns encountered during this project.

10.5.1 Migration Between Server 2019 Hosts Preserving TPM and Secure Boot

This scenario applies when a Windows 11 VM must retain its full Windows 11 hardware requirements while being moved between two Windows Server 2019 Hyper-V hosts. The VM is exported from the source host, then re-created on the destination host as a Generation 2 VM using the existing exported system disk. TPM is re-enabled by assigning a new local key protector, as the original key protector cannot be reused on a different host.

1. Export the VM from the source host
Shut down the VM and perform a standard export to a location accessible from the destination host.
2. Create a new VM shell on the destination host
Create a new Generation 2 VM with the same name and matching hardware settings, but do not enable TPM and do not create a new system disk. Instead, attach the exported VHDX as the VM's system disk.
3. Reassign the key protector and enable TPM
In an elevated PowerShell window on the destination host, run these commands
`Set-VMKeyProtector -VMName "VM-NAME" -NewLocalKeyProtector`
`Enable-VMTPM -VMName "VM-NAME"`
4. Start the VM and confirm normal operation

10.5.2 Migration of Non-Compliant Windows 11 VM Between Server 2019 Hosts

This scenario applies when a Windows 11 virtual machine no longer enforces Windows 11 hardware requirements such as TPM and Secure Boot, and there is no intention to re-enable them after migration. Because the Windows 11 upgrade checks are only enforced during installation or upgrade, an already-installed Windows 11 VM with these features disabled can be migrated like any standard Generation 2 virtual machine. As long as both Hyper-V hosts are running Windows Server 2019 Datacenter, the VM can be moved using the built-in export and import process.

1. Prepare the VM on the source host
Shut down the VM and confirm that TPM and Secure Boot are disabled. If changes are made, perform a quick boot-login-shutdown test before exporting.

2. Export the VM
Export the virtual machine from the source host using Hyper-V Manager.
3. Import the VM on the destination host
Use Hyper-V Manager's Import Virtual Machine wizard to complete the migration and start the VM.

10.5.3 Migration From a Windows Server 2019 Host to a Windows Server 2016 Host

This scenario applies when a Windows 11 virtual machine created on a Windows Server 2019 Hyper-V host must be moved to a Windows Server 2016 host. In addition to supporting an older virtual machine configuration version, Windows Server 2016 has limited support for TPM-backed key protectors and Secure Boot certificates, making it impractical to preserve Windows 11 hardware enforcement during migration. As a result, the VM must be migrated without TPM and Secure Boot by re-creating the virtual machine on the destination host and reusing the existing system disk.

1. Prepare the VM on the source Server 2019 host
Shut down the VM and disable Secure Boot and TPM in the VM's Security settings and then export the VM to a location accessible from the Server 2016 host.
2. Create a new VM shell on the Server 2016 host
Copy the exported VHDX to the destination host. Create a new Generation 2 VM and select Use an existing virtual hard disk, pointing to the copied VHDX.
3. Verify settings and start the VM
Open the VM's Security settings to confirm Secure Boot and TPM remain disabled. Adjust any remaining settings to match the original configuration, then start the VM and confirm normal operation

10.6 Checklist: Reducing VHDX size for Windows 10 VMs

Shrinking a dynamically expanding VHDX requires preparing the guest OS so that Hyper-V can safely reduce the virtual disk's maximum size. Windows normally protects certain system files and partitions from being moved or deleted, so the steps below disable those protections temporarily, shrink the C: volume, and then shrink and compact the VHDX from the host. These steps must be followed **exactly in order**, as each action enables the next.

Phase 1 — Prepare the disk from inside the guest VM

The goal of this phase is to remove or disable components that “pin” data to the end of the disk. As long as any of these are present, Windows will refuse to shrink the volume.

1. Disable system components that prevent shrinking

Disable the pagefile. Open System Properties → Advanced → Performance → Settings → Advanced → Virtual memory.

Select No paging file, click Set, then OK, and reboot if prompted.

Disable hibernation. The hibernation file (hiberfil.sys) is large and fixed in location and it also blocks shrinking. Open Command Prompt as Administrator and run:

```
powercfg /hibernate off
```

Disable Windows Recovery Environment (WinRE). WinRE owns the final recovery partition at the end of the disk, which blocks volume shrink operations until removed.

Open Command Prompt as Administrator and run:

```
reagentc /disable
```

This makes the last recovery partition removable.

2. Delete the last recovery partition

The final recovery partition must be removed before the C: volume can be shrunk to the desired size. From command prompt, open DiskPart and remove the last partition at the end of Disk 0

```
diskpart
list disk
select disk 0
list partition
select partition X    ← select the last small recovery partition
delete partition override
exit
```

At this point, the end of the disk should consist of free space only.

3. Clean and consolidate disk space

This step reduces the amount of used space and moves files toward the beginning of the disk.

- Run Disk Cleanup *as Administrator*
Select temporary files, Windows Update cleanup, Recycle Bin, and similar items.
- Open Defragment and Optimize Drives Optimize the C: volume

4. Shrink the C: volume

Hyper-V cannot shrink the VHDX smaller than the largest partition inside it. The C: volume must be reduced first. This defines the *logical* size of the Windows disk before Hyper-V modifies the VHDX.

Open Disk Management, right-click the C: volume, and choose Shrink Volume. Enter the shrink amount so that the C: volume fits the desired final size (for example, shrinking to ~68 GB used).

Do not proceed unless the shrink operation completes successfully.

Phase 2 — Shrink and compact the VHDX from the Hyper-V host

At this stage, Windows sees a smaller disk, but the VHDX file itself is still large. These steps reduce the virtual and physical size.

5. Shrink the virtual disk

Shut down the VM completely.

In Hyper-V Manager, open `Settings → Hard Drive → Edit`, choose Shrink, and enter the new disk size (for example, 75 GB).

This aligns the VHDX maximum size with the shrunk guest volume.

6. Compact the VHDX

In the same Edit wizard select Compact and complete the wizard.

Compaction reclaims unused physical blocks inside the VHDX file. After this step, the VHDX file on the host should physically shrink. Shrinking reduces the maximum size; compacting reduces the actual physical file size on disk. Both steps are required.

Phase 3 — Restore minimum functionality inside the guest VM (after restart)

The VM needs a pagefile for normal operation. Hibernation and System Restore do not need to be re-enabled unless required.

7. Re-enable the pagefile

Return to the Virtual Memory settings and select “System managed size”, click Set, then OK.

8. Verify final disk layout

Open **Disk Management** and confirm:

- Disk 0 reflects the expected size (for example, ~75 GB)

- No unallocated space or leftover partitions exist at the end of the disk

This verification step confirms that both the guest OS and the VHDX are correctly aligned, the shrink was successful and the VM is using the new disk boundaries correctly.

Additional Notes

- Hibernation and WinRE do **not** need to be re-enabled unless there is a specific operational requirement.
- As long as guest disk usage remains below the virtual disk maximum, the VHDX file will not grow beyond its reduced size.
- If the VM later grows close to the new limit, the VHDX will expand again (up to the new maximum), but it will not exceed it.
- If any step fails, **do not proceed**; resolve the issue before continuing, as later steps depend on earlier disk state changes.

11. Linux as a First-Class Desktop Platform

Linux Mint was initially selected to extend the life of several laptops that could not meet Windows 11 hardware requirements. During evaluation, Linux proved fully capable of offering strong performance, stability, and ease of administration. As a result, Linux has been embraced as a first-class, co-equal desktop platform, enabling the organization to move beyond a Windows-only model while maintaining compatibility with existing Windows-based infrastructure.

The converted systems must support a core set of representative use cases to ensure continuity of operations.

- **User Login and Desktop Access** – the converted system must allow a user to log in and access a functional desktop environment with working input and display devices.
- **Network Connectivity and Remote Desktop Access** – the converted system must support stable RDP connections to key Windows-based systems used by sys admins and developers, connecting to other computers on both the primary wired network and the secondary Wi-Fi network. The converted system itself should also be accessible remotely through the network.
- **File System Operations** – the converted system can perform basic file management tasks on local and network drives.
- **Text File and Office Document Editing** – the converted system can edit and save plain text files using a graphical editor, and can edit and save Word and Excel documents.
- **Web Browsing** – the converted system can access internal and external websites through a web browser.
- **Audio and Video Usage** – the converted system can play media, records audio, and support video conferencing.

11.1 Installing Linux Mint

To begin transforming a Windows 10 Pro laptop into a Linux Mint-based RDP client, the existing Windows operating system must be replaced. This is typically done using a bootable USB stick prepared with the Linux Mint installer. After downloading the latest installation ISO from linuxmint.com, use a third-party tool such as Rufus to burn the ISO into a USB stick. The physical laptop is booted directly from USB, and the installation proceeds through Mint's graphical setup tool. During this process, the installer does not prompt the user to assign a hostname or configure network settings beyond basic Wi-Fi support. These tasks must be handled post-

installation once the system is up and running. At this stage, Linux Mint becomes the primary operating system on the laptop, ready to be configured for remote access tasks.

11.2 Fixing Network and Hostname Configuration

After installation, the Ethernet interface must be verified and configured to properly integrate with the organization's LAN, including using the internal DNS servers hosted on domain controllers. This ensures the system can reliably resolve both internal and external domain names and participate effectively in local network operations. To configure this:

1. Open the Network Settings from the system tray and disable Wi-Fi to prioritize the wired Ethernet connection.
2. Edit the active wired connection and retain the IPv4 method as “Automatic (DHCP)” to allow dynamic IP assignment. Avoid static IPs to preserve flexibility and reduce administrative overhead.
3. Override the default DNS settings by manually entering the domain controller IP addresses along with the local router: 192.168.0.91, 192.168.0.92, and 192.168.0.1. This prevents the system from relying on inaccurate defaults.
4. Save the changes and reboot the system to apply new network settings.

To complete the foundational setup, assign the correct hostname to the laptop using the following command:

```
sudo hostnamectl set-hostname SOL
```

Replace SOL with the appropriate system name as needed. This sets the internal identifier used by the system and ensures alignment with the naming conventions used throughout the infrastructure.

11.3 Outbound RDP Connections – Remmina

To provide outbound RDP connectivity from Linux Mint to Windows workstations, the Remmina remote desktop client must be installed. Remmina is available in the standard Mint repositories and supports a variety of remote protocols including RDP, SSH, and VNC. Installation should include both the core application and the RDP plugin.

Install Remmina using the following command in the terminal:

```
sudo apt update  
sudo apt install remmina remmina-plugin-rdp -y
```

Once installed, launch the application from the Mint menu and create a new connection profile. This profile simplifies future use by associating a machine name with its connection details.

To create a connection to a Windows machine on the LAN, enter the IP address (e.g., 192.168.0.93), along with the local Windows username (e.g., Administrator). The domain field

should be left blank for machines that are part of a WORKGROUP rather than an Active Directory domain. Once the profile is saved, it can be reused with a single click, providing stable, full-screen remote access functionality similar to Windows RDP.

11.4 Inbound RDP Connections – XRDP

To install the XRDP service to allow inbound RDP sessions from windows computers

```
sudo apt update
sudo apt install xrdp -y
```

To enable and start XRDP on a Linux Mint computer

```
sudo systemctl enable xrdp
sudo systemctl start xrdp
```

To verify it's running

```
sudo systemctl status xrdp
```

Potential security concern

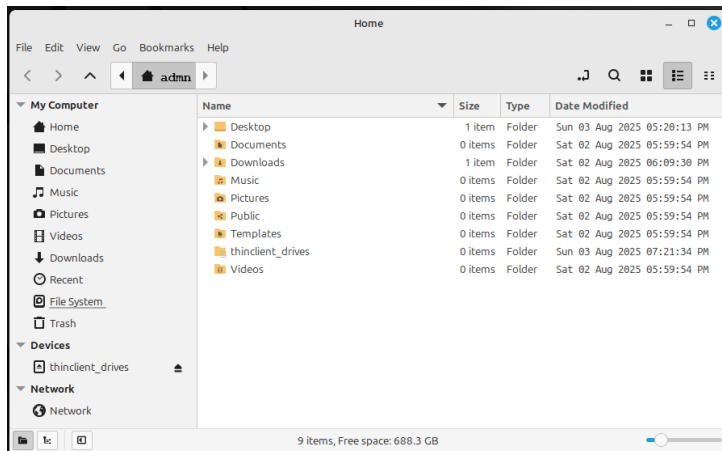
Xrdp opens inbound port 3389 for inbound rdp connections. If the Linux Mint computer does not have a firewall such as UFW, and it connects to the Wi-Fi subnet, then a compromised device connected to the Wi-Fi subnet could attack the wired corporate network through the Linux Mint computer. Therefore in order to connect the Linux Mint computer to Wi-Fi, UFW must be installed and configured with an inbound rule blocking port 3389 from being accessed through the Wi-Fi subnet.

For this reason, Wi-Fi is not being set up for the Linux Mint computers at this time. A project scope extension will be needed for the following tasks

- Install UFW – this is a firewall for Linux Mint
- Install GUPFW – firewall configuration GUI
- Use GUPFW to configure UFW to block all inbound connections from Wi-Fi
- Connect to Wi-Fi

11.5 Accessing the File System in Linux Mint

Linux Mint uses a file manager called Nemo, which serves as the equivalent of Windows File Explorer. It provides a graphical interface for browsing the local file system, launching applications, and accessing network resources.



To open Nemo, click the Files icon from the Start menu or taskbar. The interface is similar to Windows Explorer, with a sidebar for quick access to common locations and a main pane for folder contents. At the top, the location bar can be toggled into a text field by pressing Ctrl + L, allowing users to enter exact paths. For example, typing `smb://file-01/MyShare` will connect to a Windows file share named MyShare on the server file-01.

When connecting to a network share, Nemo may prompt for credentials. Users should select “Forget password immediately” unless they explicitly wish to save credentials for future sessions. Once connected, the share will appear in the sidebar and may also temporarily appear on the desktop as a mounted volume. These desktop icons disappear when the share is unmounted or the system is rebooted.

Nemo supports double-click launching of files, and integrates with applications such as LibreOffice for documents and Image Viewer for media. It is the recommended tool for file navigation and network access on Linux Mint workstations.

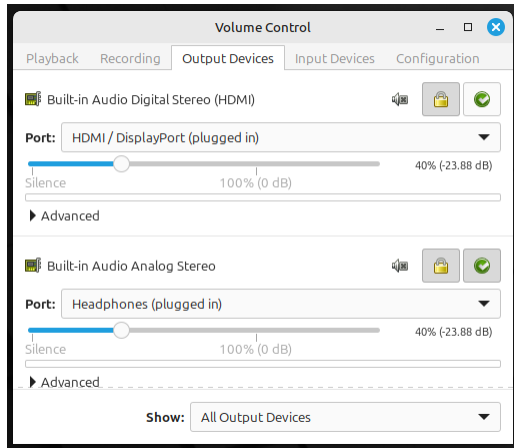
If credentials appear to be saved despite the “Forget password immediately” setting, users can force the system to forget them by deleting all files in the keyring directory. This can be done by opening a terminal and running

```
rm ~/.local/share/keyrings/*
```

which removes any saved passwords or authentication tokens associated with the current user. To ensure that no credentials remain cached in memory, it is recommended to reboot the system after performing this step. Once restarted, users can verify that the credentials have been cleared by reconnecting to the network share using Nemo; if prompted again for a username and password, the previous credentials have been successfully forgotten.

11.6 Managing Sound

To enable sound input and output on Linux Mint, we installed a set of packages that manage audio devices and provide recording functionality. The primary tool for device management is PulseAudio Volume Control,

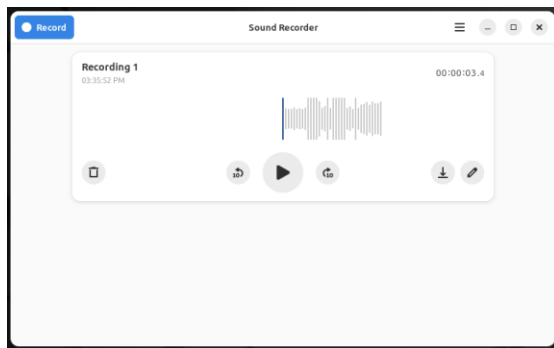


installed via the command:

```
sudo apt install pavucontrol
```

This utility offers a graphical interface for configuring microphones, speakers, and other audio hardware.

For recording and playback, we used GNOME Sound Recorder,



installed with:

```
sudo apt install gnome-sound-recorder
```

These are referred to as packages in Linux terminology, though they may also be described as applications when used in a desktop environment.

By default, GNOME Sound Recorder stores audio files in a hidden directory located at

```
~/.local/share/org.gnome.SoundRecorder/
```

This path is not immediately visible in the file manager unless hidden files are revealed (typically with Ctrl + H). To simplify future access, we recommend creating a desktop shortcut to this folder once it's been located.

11.7 Linux Mint Backup and Restore with Timeshift

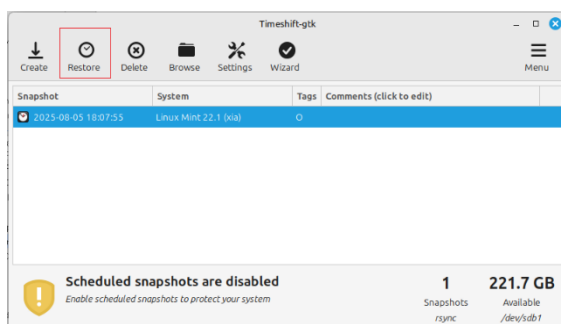
Reliable backups are essential for maintaining system stability, especially when making configuration changes, installing new software, or applying updates. In our Linux Mint deployment, we rely on Timeshift, which comes pre-installed as part of the distribution. It provides snapshot-based backups that preserve both system files and user data, allowing us to revert to a known-good state if needed.

Timeshift comes pre-installed with Linux Mint and is fully integrated with the system. During snapshot creation, it is critical to ensure that personal files are included by selecting the /home option from the dropdown in the setup or snapshot screen. By default, Timeshift excludes user data under /home, which would result in an incomplete backup. Snapshots were manually triggered before major changes, serving as milestone checkpoints in our workflow.

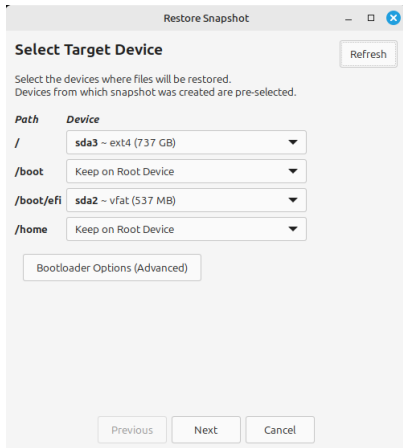
Restoring from a snapshot can be unintuitive. After rebooting, Timeshift may fail to restore user data unless specific steps are followed. While testing restores, we encountered failures where the process silently aborted after the "dry run" phase, especially immediately after rebooting the system. Repeating the restore a few times typically resolved the issue. We also identified some other specific actions that made success more likely. We discovered that opening the Bootloader Options (Advanced) dialog and immediately closing it without making any changes often triggers the full restore wizard. Additionally, on the Target Device screen, change the /home dropdown from "Keep on Root Device" to the actual system partition, e.g. "sda3 – ext4 (737 GB)". Once these steps were followed, the restore succeeded and returned the system to the exact state captured in the snapshot.

Steps to Restore a Snapshot Successfully

After launching Timeshift and selecting the desired snapshot, follow these steps.



1. Click Restore.
2. On the Target Device screen, change /home from “Keep on Root Device” to the correct partition.



3. Click Bootloader Options (Advanced) and then Close the dialog without making changes.
4. Click Next and verify that the Confirm Actions screen appears.
5. Click Next again to begin the restore process.
6. After completion when the system has rebooted, confirm that both system and user data have been restored.

For step 2, to identify the correct partition that currently holds /home, run the following command in a terminal:

```
df -h /home
```

This will return output similar to:

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda3	687G	12G	640G	2%	/

Look for the partition where the MOUNTPOINT is / this is your root system.

If the restore silently fails, repeat the steps. Sometime multiple tries are needed, but eventually it should work.

11.8 Securing Wi-Fi in Linux Mint Using a Firewall

The wi-fi subnet has a larger attack surface than the company's internal domain which is behind a wired router. Since Linux Mint doesn't come with Windows Defender Firewall, we need to use a third-party firewall to secure the Linux Mint laptops when exposing them to the wi-fi subnet.

A simple, easy-to-use firewall available for Linux Mint is ufw and gufw. These two packages should come with Linux Mint by default. But if you need to install them, run the following in a Linux Mint terminal.

```
sudo apt install ufw
sudo apt install gufw
```

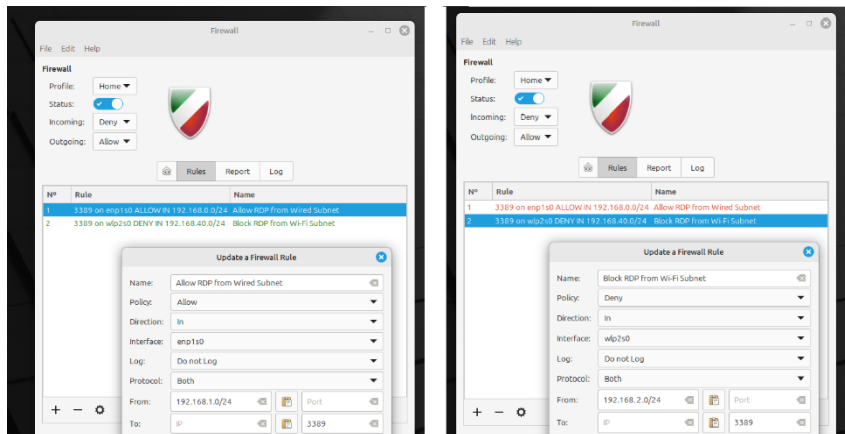
This installs the Uncomplicated Firewall (UFW) and its graphical user interface (GUFW), which together provide a simple firewall experience similar to Windows Defender Firewall.

Next, enable them.

```
sudo ufw status
sudo ufw enable
```

To further harden the laptop's network posture while connected to both wired and wireless interfaces, GUFW, the graphical frontend for UFW, can be used to define interface-specific firewall rules. This approach allows us to selectively permit or deny Remote Desktop Protocol (RDP) traffic based on both the source subnet and the network interface.

The rules are created using the Advanced tab in GUFW, which provide granular control beyond what is available in the Preconfigured or Simple tabs. This tab allows us to explicitly define the source IP range, destination port, interface and protocol to which the rule applies. The use of GUFW's Advanced tab provides a low-code, GUI-driven method for implementing precise firewall rules without requiring terminal commands or manual edits to configuration files.



We only need to define inbound rules, since the goal is to protect the laptop from unsolicited traffic originating from the unsecured Wi-Fi segment of the network. Outbound traffic from the laptop is already governed by default policies that are appropriate for our use case, and we are explicitly allowing trusted computers on the secure wired subnet to initiate connections. This

approach ensures that the laptop remains accessible to authorized systems while remaining shielded from potentially hostile traffic.

Additionally, we specified the appropriate network interface for each rule—either the wired interface (enp1s0) or the wireless interface (wlp2s0)—to ensure that the firewall behavior was correctly scoped to the physical connection in use. This configuration allows us to enforce differentiated access controls based on network topology, ensuring that RDP traffic is permitted only from trusted sources and blocked from untrusted ones.

11.9 Regression Tests for a Linux Mint Workstation

11.9.1 User Login and Desktop Access

Log in to the laptop using a valid user account and confirm that the desktop environment loads successfully. Verify that the keyboard, mouse, and display are responsive and functioning.

Expected: The login completes without error, the desktop appears within 10 seconds, and the keyboard, mouse, and display all respond promptly to user input.

11.9.2 Wired Network Access and RDP Connectivity

Connect the laptop to the primary wired network using an Ethernet cable. Confirm that the system obtains a valid IP address and can access internal and external websites. From another system on the wired network, attempt to connect to the laptop using RDP.

Expected: The laptop connects to the wired network within 10 seconds and can browse websites. RDP connections from other systems on the wired subnet are accepted.

11.9.3 Wi-Fi Network Access and RDP Blocking

Connect the laptop to the secondary Wi-Fi network using the designated SSID and credentials. Confirm that the system obtains a valid IP address and can access internal and external websites. From another system on the Wi-Fi network, attempt to connect to the laptop using RDP.

Expected: The laptop connects to the Wi-Fi network within 15 seconds and can browse websites. RDP connection attempts from other systems on the Wi-Fi subnet are blocked by the firewall.

11.9.4 File System Operations

Open the file manager and navigate to the local file system (e.g., /home/sysadmin/) and a mounted network share (e.g., smb://MyFileServer/MyShare/). Create a new folder in each location and place a test text file inside.

Expected: The system successfully creates folders and files in both the local directory and the network share. No permission errors or delays occur during file creation, access, or deletion.

11.9.5 Text File and Office Document Editing

Open a plain text file (e.g., /home/sysadmin/test.txt) using a text editor such as Xed or Gedit. Make a small edit, save the file, and verify the changes persist. Then open an office document (e.g., .docx or .xlsx) using LibreOffice Writer or Calc, make a minor edit, save, and reopen to confirm the changes.

Expected: The system successfully opens, edits, and saves both text and office document files. Changes persist after closing and reopening the files, and no errors or unexpected behavior occur during the editing or saving process.

11.9.6 Web Browsing

Open the default web browser (such as Firefox or Chrome), navigate to a well-known external website (e.g., <https://www.google.com>), and verify that the page loads fully. Next, navigate to an internal company web resource, such as the intranet homepage or internal documentation site, and verify successful loading and access.

Expected: Both external and internal websites load completely without errors, and basic browsing functions such as clicking links and navigating pages work as intended.

11.9.7 Audio and Video Usage

Play a video with sound (for example, a YouTube video) using the default media player or web browser and verify that audio plays clearly through the speakers. Next, use a simple sound recording application to record a short audio clip via the built-in microphone and then play back the recording to verify audio capture and playback. Finally, launch a video conferencing application (such as Zoom or Teams) and verify that both video and audio are functioning during a test call or meeting.

Expected: Audio playback is clear and audible through local speakers; audio recording captures sound correctly and playback functions properly; video conferencing shows both audio and video without issues.

11.9.8 Printing

Open a document and initiate a print job using the system's default print dialog. Select a known working printer from the list and send the job. Confirm that the printer receives the job and produces a physical copy with correct formatting and content.

Expected: The system successfully connects to the selected printer, sends the print job without error, and produces a correctly formatted physical document. The print dialog displays available printers, and no driver or connectivity issues occur during the process.

12. Operational Standards

Below are concise operational checklists used during migration and any future setup:

12.1 Standards Followed

Where practical, we adhere to the following industry and internal standards when managing systems (however we favor a pragmatic rather than a dogmatic approach)

- Microsoft Best Practices for Active Directory, DNS, and file sharing security.
- Principle of Least Privilege for all file share and administrative permissions.
- Hyper-V Management Guidelines to ensure safe VM exports and imports.
- Repeatable Setup using internal checklists tailored to our domain structure (d1.internal) and decentralized Hyper-V hosts.
- Workgroup Isolation Policy: All Hyper-V hosts remain in WORKGROUP mode for administrative simplicity and reduce domain dependency at the virtualization layer.

12.2 Repeatable Setup Checklists

File Share Permissions

- Use "Everyone: Full Control" at the share level (if not sensitive).
- Use NTFS-level permissions for actual access control. (Specific permissions are determined by operational needs.)

AD-Integrated DNS Setup

- Enable dynamic updates (secure only).
- Define forwarders to public resolvers (e.g., 8.8.8.8, 1.1.1.1).
- Replicate DNS zones to:
 - dc-01
 - dc-02
- To verify reverse lookup zones exist:

```
Get-DnsServerZone -ComputerName dc-01 | Where-Object { $_.IsReverseLookupZone }
```

Domain Join and Workstation Profile Transition

- Backup local user data (Desktop/Documents/Downloads).
- Join workstation to d1.internal domain.
- Log in once with domain user to create profile.
- Reconfigure network drives and printers manually.

Scheduled Task Migration

Each task scheduled on lunar-batch was updated with the appropriate domain account username and password.

Hyper-V Export and Disaster Recovery Preparation

Exporting both domain controllers together (one-time disaster recovery snapshot):

1. Shut down both DCs via Hyper-V Manager.
2. On each hypervisor host:
 - Right-click the VM > Export.
 - Target: External volume (e.g., E:\VM-Exports\YYYYMMDD\).
3. Confirm export folder includes the full VM config, VHDs, and snapshots (if any).
4. Label folders with computer name and export date.

For all other hyper-v vm exports, follow company policy.

13. Testing and Validation

Always test everything! – well known saying

13.1 Domain Resilience Tests

- Each domain controller was shut down individually to verify that authentication, DNS resolution, and Group Policy processing continued without interruption.
- Verified login success from domain-joined workstations during each simulated DC outage.

13.2 Post-Migration Validation

- Confirmed domain join success and user profile creation on migrated workstations.
- Verified DNS resolution using nslookup for local and external domains.
- Tested file share access and permissions using domain credentials.
- Confirmed TFS, SQL Server, and IIS services were reachable and functional.

13.3 Layered Testing

Layered testing refers to progressively testing systems and configurations at increasing levels of complexity and scale to reduce risk and catch issues early. Each layer reduced uncertainty before moving on to the next, minimizing the blast radius of potential errors. It's a phased approach to validation.

Various different variations of layered testing were used in this project. Examples of layered testing that were used: WSUS proof-of-concept, domain migration.

13.3.1 WSUS Proof-of-Concept – Layered Testing

1. Initial Setup Validation:
 - Confirmed WSUS2 could be installed and configured using WID and IIS.
 - Verified default update classifications and product categories.
2. Domain Join and Communication Check:
 - Joined a single test system (win2016-5) to the d1.internal domain.
 - Applied local group policy and confirmed registration with WSUS2.
3. Approval and Delivery Simulation:
 - Approved a Windows Update and ensured no direct downloads from Microsoft.
4. Update Visibility and Targeting:
 - Tested client group membership targeting on selected updates.
5. Installation and Reporting:
 - Installed update via GUI (no forced CLI tools) and confirmed status report.

13.3.2 Domain Migration – Layered Testing

- Layered environment testing occurred by building and validating the d1.internal domain in isolation before migrating any production systems.
- Hyper-V rebalancing of virtual machines between hypervisors tested on cloned virtual machine exports.
- Staged migration of machines allowed validation of DNS, AD authentication, and file share access step-by-step.

14. Outstanding Tasks and Future Plans

The following items were intentionally deferred or identified during the project as opportunities for future improvement, testing, or documentation.

14.1 TFS Migration to Git repo in Azure DevOps

To modernize version control and simplify access, all Team Foundation Server (TFS) projects on lunar-tfs will be migrated to Git repositories hosted in Azure DevOps. This will improve developer experience, support remote collaboration, and align with industry trends.

14.2 Verification and Testing of webserv-test under d1.internal

Conduct end-to-end validation of webserv-test to confirm post-migration functionality:

1. Ensure the server is joined to d1.internal and rebooted.
2. Confirm SQL connectivity using Invoke-Sqlcmd or equivalent testing via SSMS.
3. Validate IIS application pools are using domain service accounts or ApplicationPoolIdentity, and that identity permissions are still valid.
4. Access hosted websites and confirm correct functionality.
5. Check Windows Firewall rules via GUI (Network and Sharing Center → Windows Defender Firewall → Advanced Settings).

14.3 Implement Speech Recognition on Linux Mint Workstations

To improve accessibility and reduce time spent on manual text input, evaluate and implement speech-to-text and screen-reading solutions for Linux Mint laptops. Candidate solutions should be tested for accuracy, integration with existing applications, and minimal performance impact.

14.4 Linux Mint Software and Peripheral Validation

To ensure Linux Mint workstations fully meet user needs, additional validation of optional software packages and peripheral hardware is recommended. This includes testing webcams, printers, and other accessories that may be required for conferencing or remote access. Evaluating compatibility and usability will help standardize configurations and reduce support overhead.

14.5 Moving the Canon Printer to Linux

As part of the broader infrastructure realignment, the Canon TS5300 printer currently hosted on TITAN should be migrated to SATURN, a Linux Mint computer. This change will reduce reliance on TITAN, which will be reserved for other roles.

SATURN will be configured to host the printer using the Common Unix Printing System (CUPS), with the printer physically connected via USB and shared over the network using Samba. Linux printer drivers will be installed on SATURN to ensure proper operation and compatibility with CUPS.

When printing from Windows clients to a printer hosted on a Linux system, the client machines do not require Linux drivers. Instead, Windows uses its own Canon TS5300 driver to generate print jobs, which are then transmitted to the CUPS server on SATURN. CUPS interprets and processes these jobs using its own Linux-based drivers. As long as the printer is correctly shared and advertised over the network, and the drivers on both ends are compatible with the printer's capabilities, cross-platform printing should function reliably without requiring Linux drivers on Windows clients.

This migration will simplify cross-platform support, reduce administrative overhead, and improve service reliability.

TROUBLESHOOTING GUIDE APPENDIX

This appendix provides supplementary materials for troubleshooting purposes.

A1 WSUS Troubleshooting

This section outlines known issues encountered during the WSUS rollout and their resolutions.

Issue: Client Shows "You're up to date" but No Updates Installed

Observed on: webserv-test.d1.internal

Symptoms:

- Windows Update (Settings app) reports "You're up to date"
- Control Panel > View Installed Updates shows no recent cumulative updates (e.g., KB5061010)
- WSUS console displays status:
 - "Not yet reported"
 - Tooltip: "This computer has not yet contacted the server"

Likely Cause:

- Windows Update client service (wuauserv) was not started after resetting the SoftwareDistribution folder.
- The client did not contact WSUS due to pending service initialization.

Resolution:

Reboot and manually start or restart Windows Update services and initiate a scan:

```
net stop wuauserv
net start wuauserv
Start-Service BITS
Usoclient StartScan
```

Once these steps are completed, the client typically appears in the correct computer group in WSUS, and pending updates will become visible.

Note: This fix may be necessary after manually clearing C:\Windows\SoftwareDistribution.

Special Case – WSUS Server as Client:

If the WSUS server itself is configured as a client:

- Use its own DNS name in the WSUS URL (e.g. `http://wsus2.d1.internal:8530`)
- Avoid using localhost or 127.0.0.1
- Follow the same steps as above

After uninstalling an update for test purposes, a reboot and manual scan may be required before WSUS detects the update as needed.

Troubleshooting issues with WSUS Client Registration and Communication

If a client fails to appear in WSUS or does not receive updates, use the checklist below.

Confirm Group Policy Application

On the client, run:

```
gpresult /r
```

If a client fails to appear in WSUS or does not receive updates, use the checklist below.

Confirm WSUS Registry Settings

```
HKEY_LOCAL_MACHINE\Software\Policies\Microsoft\Windows\WindowsUpdate
```

Expected values:

- WUServer = `http://wsus2.d1.internal:8530`
- WUStatusServer = `http://wsus2.d1.internal:8530`

If missing, verify local Group Policy and re-apply with `gpupdate /force`.

Trigger WSUS Communication

If the client doesn't check in automatically, try the following on the client

```
wuaclt /detectnow  
wuaclt /reportnow  
net stop wuauserv  
net start wuauserv
```

Note: wuaclt is deprecated on Windows 10/11 but still works on Server 2016. Do not use usoclient unless troubleshooting Windows 10/11 specifically.

Check WSUS Server Health

- Verify WsusPool is running in IIS. If not, verify non-default memory settings (see 9.4).
- Do a manual synchronization in WSUS console to confirm the server can pull updates.
- If client remains under Unassigned Computers, manually assign it to the correct group.

WindowsUpdate.log Review

On the client, run

Get-WindowsUpdateLog

This generates a log file on the Desktop. Search for lines referencing wsus2.d1.internal to confirm the correct server is being contacted.

Issue: Scheduled Tasks Fail to Trigger After VM Startup

Observed on:

eu-web.d1.internal

Symptoms:

- Scheduled task appears in Task Scheduler history with status “Missed task start rejected”

Likely Cause:

- The VM was not fully initialized when the task was scheduled to run

Resolution:

- Reschedule task at least 15 minutes after VM startup
- Enable “Run task as soon as possible after a scheduled start is missed” in task settings

Notes:

- This scenario is common on slower hypervisors such as Europa

Issue: WSUS Scan Fails Despite Script Execution

Observed on:

eu-web.d1.internal

Symptoms:

- forcewuscan.ps1 logs successful start/finish
- WindowsUpdate.log shows timeout or 0x8024401c errors

Likely Cause:

- WSUS server was offline or non-responsive during scan attempt

Resolution:

- Confirm WSUS availability via browser or ping before scan time
- Restart IIS on the WSUS server and verify WsusPool is running

Issue: Updates Install but Auto-Restart Never Occurs

Observed on:

IIS web servers eu-web and webserv-test

Symptoms:

- Update listed as “Requires a restart to finish installing”
- Uptime exceeds 24 hours with no reboot Likely Cause:
- Active IIS services prevent Windows from reaching idle threshold for automatic restart

Resolution:

Manually restart the WSUS client computer.

Issue: Windows Update Corrupted

Symptoms:



Resolutions:

1. Reboot and try again.
2. Run Windows Update Troubleshooter and try again.
3. Delete the `SoftwareDistribution` folder and try again. This resolution is complicated by the fact that Windows services constantly use this folder, so you need to temporarily stop those services, delete the folder, and then restart the services. The services in question are the Windows Update service (WUAUSERV) and the Background Intelligent Transfer

Service (BITS). Here are the steps to delete the folder, after which you can try the windows update again.

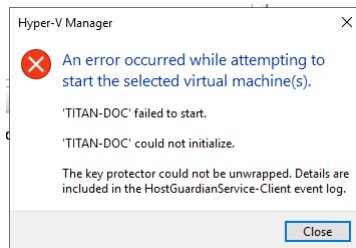
```
net stop wuauserv
net stop bits
delete C:\Windows\SoftwareDistribution
net start wuauserv
net start bits
```

A2 Windows 11 Troubleshooting

This section of the appendix provides troubleshooting for the Windows 10 to Windows 11 replacement initiative.

Resolving a Corrupted vTPM That Blocks VM Startup

Error: "The key protector could not be unwrapped."



Problem: When enabling Secure Boot and virtual TPM on the TITAN-DOC VM, it refuses to start with the error "The key protector could not be unwrapped."

At the same time, Windows 11 Setup will not proceed without TPM 2.0 enabled—creating a catch-22 that blocks both normal VM startup and in-place upgrade.

Root Cause: The VM's virtual TPM (vTPM) metadata or associated key protector had become corrupted. This prevented Hyper-V from initializing the vTPM and caused the VM to crash when attempting to boot with TPM enabled.

Solution Steps:

1. Preserve the original virtual disk: Shutdown and rename the broken VM (e.g. TITAN-DOC → TITAN-DOC-OLD).
2. Make a copy of its virtual hard drive (e.g. copy C:\Imports\TITAN-DOC\Virtual Hard Disks\MERCURY-DOC.vhdx to a temp location such as the desktop)

3. Create a clean clone VM: in Hyper-V Manager, create a new Generation 2 VM (e.g. TITAN-DOC-CLONE) matching CPU, memory, Secure Boot, and firmware settings, but do not attach the OS disk yet.
Leave TPM disabled during creation.
4. Move the copy of the original virtual hard drive file from the temp location into the new VM's Virtual Hard Disk folder.
5. Attach the copy of the original disk: under SCSI Controller > Hard Drive, browse to and select the copy of the original virtual hard drive file from Virtual Hard Disk folder.
6. Reset the vTPM protector on the Hyper-V host using PowerShell (run as Administrator):

```
Set-VMKeyProtector -VMName "TITAN-DOC-CLONE" -NewLocalKeyProtector  
Enable-VMTPM -VMName "TITAN-DOC-CLONE"  
Start-VM -Name "TITAN-DOC-CLONE"
```

7. The cloned VM started successfully with TPM enabled.
8. Log in to confirm the OS, domain membership, and applications are intact.

This solution avoids any changes to Hyper-V host security or guarded fabric features, and sidesteps vTPM corruption entirely.

Microsoft Authentication MFA Infinite Loop

Error: An issue was encountered when logging in as local administrator and attempting to open Visual Studio 2017 Community Edition (which is the version used for supporting the legacy application). The Microsoft credentials were stale and could not be renewed. The multifactor authentication requiring entering a verification code sent to my email address, went into an infinite loop: the email address was confirmed, a code was sent, I retrieved the code from the email and typed it into the authentication dialog in VS correctly, and the process simply repeated (another code was emailed and entered, repeatedly). However when logging into DEVBOX-1 as domain administrator, I was able to use VS 2017 without any problems, my credentials were not challenged at all, they were somehow already in VS correctly (probably some kind of caching mechanism was used by VS). Why would it work fine when logged in as domain administrator, but go into an infinite loop when logged in as local administrator? Could Active Directory be somehow interfering with the authentication to my online Microsoft account? The work around is for me to always log in to DEVBOX-1 as domain administrator when I need to work with the code. However it would be helpful to understand the root cause.

What's happening?

When you run VS under your Domain Admin session, it picks up your existing AD-issued OAuth token (via Integrated Windows Authentication) to silently acquire "Microsoft Account" tokens. Under a Local Administrator session, VS attempts a full modern auth flow against your Microsoft

account but Windows itself isn't AAD-joined, so the authentication handshake never completes and you get caught in that verification loop.

What's happening under the hood

Visual Studio 2017 uses the Web Account Manager (WAM) and the Active Directory Authentication Library (ADAL) to sign you into your work ("Microsoft") account. When you're signed in with a domain account, Windows can leverage Integrated Windows Authentication (IWA) or cached tokens behind the scenes, so you never see the MFA prompt.

When you switch to a local administrator profile, however, none of those domain-joined SSO hooks exist. VS falls back to an embedded login flow, repeatedly brokers a challenge to your tenant's AD FS/Entra endpoint, then loops when it can't fully complete IWA or cache the token properly. Domain-joined logons get Kerberos tickets and silent token refresh; local accounts do not.

Why AD "seems" to interfere

It isn't Active Directory deliberately blocking Microsoft account logins—it's the absence of AD-joined SSO for your local admin user. Your tenant's conditional-access policy or AD FS may require a specific flow (IWA, OAuth via the system browser, etc.). Without a valid SPN/Kerberos ticket or a properly registered WAM broker, the local profile gets stuck in a challenge/response oop

Workarounds / Recommendations

- Stick to your domain account for dev work in VS. That path "just works" because your d1.internal domain is trusted by Azure AD.
- If you ever must run as a local user, you can add your Microsoft identity explicitly under Settings → Accounts → Access work or school → + Connect, and then sign in there. That registers the device for modern auth flows.
- Alternatively, create a Personal Access Token in your Azure DevOps/TFS profile and use that instead of interactive sign-in.

Unless you have a hard requirement to use the local account, the simplest fix is "always open VS as your domain user."

How to fix it

1. Switch VS to use your system browser In Visual Studio: Tools → Options → Environment → Accounts → "Add and reauthenticate accounts using:" → select System web browser, then sign out/sign back in. This forces the MFA flow through Edge/Chrome, which handles modern OAuth better than the embedded control.
2. Clear stale cached credentials

- a. Open Credential Manager (Control Panel → Credential Manager → Windows Credentials).
 - b. Remove entries related to Visual Studio or Azure AD (e.g. “VS.Licenses” or “Generic Token Cache”).
3. Ensure the WAM broker is installed. In an elevated PowerShell prompt inside the VM, run:

```
# For Entra (work) accounts
if (-not (Get-AppxPackage Microsoft.AAD.BrokerPlugin)) {
    Add-AppxPackage -Register `
        "$env:windir\SystemApps\Microsoft.AAD.BrokerPlugin_cw5n1h2txyewy\Appxmanifest.xml" `
        -DisableDevelopmentMode
}
```

4. Then reboot and retry the VS sign-in.

After these steps, your local admin profile will handle the MFA handshake properly—just as smoothly as your domain admin account does.

Problems Encountered with Licensing Transfer and Upgrade Path

The “Saturn” physical laptop upgrade required a nonstandard multi-step approach due to licensing and media compatibility issues. Saturn’s valid Windows 10 Enterprise retail key was reassigned to a VM on the hypervisor “Sirius” after repurposing the physical device for Linux Mint. The original installation media (version 1511) was outdated and incompatible with direct Windows 11 upgrades—even with the unsupported hardware bypass registry entry in place.

To resolve this, the upgrade followed four critical steps:

1. Bypass Windows 10 Enterprise v1511 entirely.
2. Install Windows 10 Enterprise v22H2 via a retail-compatible ISO created using the Media Creation Tool and license key.
3. Apply the `AllowUpgradesWithUnsupportedTPMOrCPU` registry key to bypass hardware checks.
4. Perform in-place upgrade to Windows 11 Enterprise using a multi-edition ISO.

Post-upgrade activation initially failed but was resolved by manually re-entering the original key. Evaluation ISOs from Microsoft’s Evaluation Center were avoided, as they cannot be activated with retail keys or upgraded to Windows 11.

This case illustrates how legacy media and mismatched editions can block upgrade paths and highlights the need for edition-aware install media when reusing retail licenses.

Catastrophic Failure of Windows 11 and The Backup Export File is Corrupted

Error: catastrophic failure of the VM and corrupt Hyper-V export

In the event the Windows 11 VM fails or is lost, and there is a problem recovering it from the last backup, you can still create a new Windows 11 VM using the Windows 10 retail license key and the Windows installation disks, and then rebuild the computer from scratch. You can either

1. install Windows 10 and activate it with the license key, and then do a free in-place upgrade to Windows 11
2. or you can install Windows 11 directly

Note: If the key isn't accepted at install time, fall back to path #1 (install Win 10 then upgrade).

The retail Windows 10 Pro key remains a valid fallback. Microsoft keeps its activation servers alive long after an OS reaches end of support. Even if Microsoft stops issuing security updates for Windows 10, that doesn't revoke your license or prevent you from reinstalling.

Blocker on the Windows 11 upgrade for the DEVBOX-1 Gen 1 VM

We upgraded DEVBOX-1 from Gen 1 to Gen 2 to prepare it for the Windows 11 upgrade. We did this by creating a clone DEVBOX-1-CLONE as a brand-new Gen 2 VM and mapping its C: drive to a modified version of DEVBOX-1's virtual harddisk .VHDX file. We managed to get the Gen 2 clone running but somehow broke the Windows Recovery (WinRE) tools. An extreme amount of time was spent trying many different approaches but we kept running into errors. We decided to stop the work on trying to fix WinRE because the rest of the Windows 10 functionality seemed to be working. We switched our attention from repairing WinRE, to the real goal of upgrading to Windows 11. In order to resume the fix for WinRE at a later time, the following AI prompt was created.

Can you skip the WinRE repair for now?

We can safely move on to the Windows 11 upgrade without a working WinRE. Skipping the DISM/extraction steps today:

- Won't block the in-place upgrade to Windows 11
- Won't prevent normal day-to-day use of the VM
- Leaves us without "Reset this PC" or Startup Repair—but we've got an exported VM backup, so we're covered if something goes wrong

We can circle back later to restore WinRE once we're past the critical path of getting Windows 11 live.

A future prompt to finish WinRE restoration

Once we do the in-place upgrade to Windows 11, a fresh WinRE image should get laid down and registered as part of setup—so the Recovery Environment will very likely be fixed automatically. We can verify this by issuing the following command in an elevated cmd prompt.

```
reagentc /info
```

If the response is something like “Windows RE status: Enabled”, we can consider the WinRE restoration a non-issue going forward. Otherwise we will have to pick this up again.

When we’re ready to pick this up again, paste the following into ChatGPT (or here), and we’ll get a fresh step-by-step:

```
***I have a Hyper-V Generation 2 VM clone running Windows 11 on a GPT disk. The Recovery partition exists but WinRE isn't registered. I've mounted my Windows 11 ISO as D: and I have a Recovery volume on partition 4. Please provide detailed instructions to:
1. Assign a temporary drive letter to the Recovery partition.
2. Extract Winre.wim from D:\sources\install.esd (or install.wim) into that partition.
3. Register and enable WinRE with reagentc.
4. Remove the temporary drive letter.
Include both DISM (with scratch dir) and 7-Zip options in case of space or permissions issues.***
```

Alternatively, Just Leave WinRE Disabled

Given the tremendous amount of time and effort spent trying unsuccessfully to enable WinRE on the VM’s disk (the VM’s internal recovery environment), we could just leave WinRE in the disabled state permanently. We have made a backup of the VM using Hyper-V export. If it’s ever needed, we can boot WinRE from the Windows installer ISO. When you need Safe Mode, Command Prompt, or an image restore, just boot the VM from Windows 11 installer ISO and choose Repair your computer → Troubleshoot → Advanced options.

TPM Key Protector Cannot be Unwrapped

When migrating Windows 11 Gen 2 VMs (with vTPM enabled) from one Hyper-V host to another, standard export/import operations may fail due to key protector mismatches. This is expected behavior if the VM was configured with a local key protector that’s bound to the original host. Follow these steps to move the VM and restore vTPM functionality.

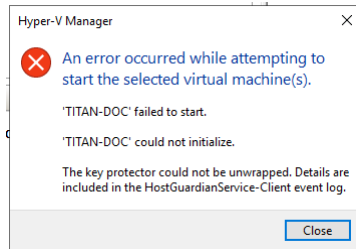
1. Export the VM on the Source Hypervisor

- Shut down the VM gracefully
 - Use Hyper-V Manager or PowerShell
- ```
Export-VM -Name "VM-NAME" -Path "D:\Exports\"
```

#### 2. Import the VM on the Destination Hypervisor

- Use Hyper-V Manager or PowerShell to import without starting
- ```
Import-VM -Path "D:\Exports\VM-NAME"
```

3. Launch Attempt Fails with TPM Error



This occurs because the TPM key protector cannot be decrypted on the new host.

4. Workaround: Create New VM Using Existing VHDX

- Create a new Gen 2 VM on the destination host
- Attach the .vhdx file from the exported VM as its system disk
- Do not enable TPM yet

5. Assign New Local Key Protector

- In elevated PowerShell on the destination host:

```
Set-VMKeyProtector -VMName "VM-NAME" -NewLocalKeyProtector  
Enable-VMTPM -VMName "VM-NAME"
```

6. Start the VM and Verify Operation

- Boot the VM normally
- Check Device Security in Windows Settings to verify TPM presence
- Run a functional test of the app workloads

A3 Linux Mint Troubleshooting

This section of the appendix provides troubleshooting for the Linux Mint introduction.

Linux Troubleshooting Tools

Terminal Equivalents to Windows Tools

Windows Command	Linux Mint Equivalent
Ipconfig	ip a or ifconfig (ifconfig may require net-tools)
ping	ping <hostname/IP> (same syntax)
tracert	traceroute <hostname/IP> (may need install)
nslookup	nslookup <hostname> or dig

netstat	ss -tuln or netstat -tuln (install net-tools)
sysdm.cpl	hostnamectl or System Info (GUI)

Linux Mint Outgoing RDP Missing

To make outgoing RDP connections from Linux Mint, you need to install Remmina.

```
sudo apt update
sudo apt install remmina remmina-plugin-rdp -y
```

Launch Remmina from the Start menu or by running remmina in the terminal.

Create a new RDP connection

- Click the + (New Connection)
- Set the following:
 - Protocol: RDP – Remote Desktop Protocol
 - Server: sirius.d1.internal or its IP
 - Username: your Windows username (e.g., d1\robidas or .\Administrator)
 - Password: (optional – can prompt each time)
- Save and double-click to connect.

Resolving a Failed Remote RDP to Linux Mint by Forced Remote Reboot

Error: RDP from a Windows computer to a Linux Mint machine fails silently, returning immediately to the local desktop with no error message.

Problem: XRDP cannot launch a remote GUI session when a local console session is active on the Linux Mint computer. The RDP client disconnects immediately without feedback.

Root Cause: Active console sessions on Linux Mint block incoming XRDP connections. If physical access to log out is unavailable, remote intervention is required.

Solution Steps:

On the Linux Mint system, install and start the SSH service:

```
sudo apt install openssh-server -y
sudo systemctl start ssh
```

From any Windows computer, open CMD or PowerShell and run:

```
ssh -t <linuxmint-username>@<linuxmint-ip-address> sudo reboot
```

Replace placeholders with the actual username and IP. Enter the password when prompted. The Linux Mint system will reboot, closing the active console session and allowing XRDP to accept future remote connections.

Security Note: SSH listens on port 22. When connecting a Linux Mint system to an untrusted or external network (e.g., Wi-Fi), configure a firewall to restrict SSH and XRDP access to trusted IP ranges.

Internet Access Lost When Moving Hypervisor to Protected Wifi Subnet

Error 1: Network icon shows that there is no internet access. Network troubleshooter reports

- There might be a problem with the driver for the Wi-Fi adapter Not fixed
- Windows couldn't automatically bind the IP protocol stack to the network adapter.

Resolution:

Open Command Prompt as Administrator and run these commands one by one

```
netsh winsock reset
netsh int ip reset
ipconfig /release
ipconfig /renew
ipconfig /flushdns
```

Then restart the computer.

Error 2: unable to find router

Resolution: Add the computer's MAC address to the router's whitelist in the advanced > network filter list.

Loss of Local Audio After RDP Connection to Linux Mint Laptop

Error: local sound stops working after connecting via RDP from a Windows 11 computer

After remotely connecting to a Linux Mint laptop (e.g., jupiter) via RDP from a Windows 11 Pro system (e.g., galaxy), users may discover that local sound playback is no longer functional. The issue does not appear immediately after the remote session ends, but becomes evident after the Linux system is restarted and logged into locally.

Cause: The issue is caused by a session startup script that launches a virtual sound redirection service for XRDP. Specifically, the file `/etc/xdg/autostart/pipewire-xrdp.desktop` is responsible for starting pipewire-xrdp, which creates a virtual playback device named `xrdp-sync`. This device is intended to route audio from the Linux session back to the remote RDP client. However, when the session is restarted locally, `xrdp-sync` can remain active or be reinitialized,

overriding the default sound output device and preventing audio from reaching the local speakers.

This behavior is problematic in environments where remote sound playback is not required, as it interferes with normal desktop audio even after the remote session has ended.

Resolution: To prevent xrdp-sync from launching during session startup, the autostart script can be safely disabled by renaming it

```
sudo mv /etc/xdg/autostart/pipewire-xrdp.desktop /etc/xdg/autostart/pipewire-xrdp.desktop.disabled
```

Renaming the file disables its execution without deleting it, allowing easy reactivation if needed in the future. This change is harmless in our environment, as we do not rely on XRDP sound redirection. Disabling the script restores normal audio routing to the local playback device and does not affect other system functions.

Testing & Verification: To confirm the fix, we reproduced the issue and verified the resolution through the following steps:

1. Initiated an RDP session from galaxy to jupiter and logged in remotely.
2. Logged out and restarted jupiter, then logged in locally and confirmed that sound playback was no longer working.
3. Renamed the autostart script using the command shown above.
4. Restarted jupiter and logged in locally again — sound playback was restored.
5. Repeated the RDP session from galaxy and verified that local sound remained functional after the session ended.

Note: This workaround disables remote sound playback during XRDP sessions, which is acceptable for our Linux Mint laptops. If remote audio becomes necessary in the future, the script can be re-enabled by renaming it back to its original filename.

Optional Tool for Diagnosing Audio Issues: pavucontrol (PulseAudio Volume Control), a graphical mixer tool for the PulseAudio sound server. It allows users to:

- View and adjust volume levels for individual applications
- Select and switch between output devices (speakers, headphones, etc.)
- Manage input devices like microphones
- Redirect playback streams to different devices mid-session It's especially useful for identifying when xrdp-sync has hijacked the default output device. You can install it via terminal:

Linux Nemo File Manager Authentication Failure

Error

When attempting to access a Windows network file share through Nemo, the following message appears

Could not display "smb://file-01/temporary/".

Error: Failed to mount Windows share: Invalid argument

Please select another viewer and try again.

[OK]

This error means that Nemo (through GVFS, the background service that handles network mounts) was unable to mount the Windows share. The phrase *“Invalid argument”* does not point to a specific problem—it simply indicates that something in Nemo’s request to the SMB client was rejected. The “Please select another viewer” part suggests Nemo has no working handler available to display the share.

Cause

This issue occurs when cached metadata or state information inside GVFS becomes corrupted or outdated, typically after a system sleep, network disruption, or configuration change. Although the share itself is reachable and DNS resolution works, Nemo fails to negotiate the mount because it is reusing stale or invalid information. In other words, the problem is not with the file server but with Nemo’s background mounting process.

Resolution

To resolve the problem, bypass Nemo’s cached state and re-establish the connection manually. This can be done with the `gio mount` command, which directly tells GVFS to mount the share using fresh parameters. For example

```
gio mount smb://file-01.d1.internal/temporary/
```

You will be prompted for credentials (domain, username, password). Once successful, the share appears again in Nemo’s sidebar and functions normally. This clears the cached or corrupt state that caused the error.

REFERENCES

- Active Directory Domain Services Overview – Introduction to AD DS
<https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/get-started/virtual-dc/active-directory-domain-services-overview>
- Active Directory Users and Computers – Managing user accounts
<https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/manage-user-accounts-in-windows-server>
- Azure Cosmos DB Emulator – Development and testing with the emulator
<https://learn.microsoft.com/en-us/azure/cosmos-db/how-to-develop-emulator>
- Azure Cosmos DB Emulator – Overview and usage
<https://learn.microsoft.com/en-us/azure/cosmos-db/emulator>
- Configures a key protector for a virtual machine
<https://learn.microsoft.com/en-us/powershell/module/hyper-v/set-vmkeyprotector?view=windowsserver2025-ps>
- DiskPart command reference
<https://learn.microsoft.com/en-us/windows-server/virtualization/hyper-v/manage/manage-virtual-hard-disks>
- DNS Client Configuration – Best practices for DNS client settings on domain controllers
<https://learn.microsoft.com/en-us/troubleshoot/windows-server/networking/best-practices-for-dns-client-settings>
- DNS Manager – Managing DNS resource records
<https://learn.microsoft.com/en-us/windows-server/networking/dns/manage-resource-records>
- Download the latest Windows 11 ISO from the official Microsoft page:
<https://www.microsoft.com/en-ca/software-download/windows11>
- Dynamic DNS Updates – How DNS dynamic updates work
<https://learn.microsoft.com/en-us/windows-server/networking/dns/dynamic-update>
- Enables TPM functionality on a virtual machine
<https://learn.microsoft.com/en-us/powershell/module/hyper-v/enable-vmtpm?view=windowsserver2025-ps>
- Generation 2 virtual machine security settings for Hyper-V
<https://learn.microsoft.com/en-us/windows-server/virtualization/hyper-v/learn-more/generation-2-virtual-machine-security-settings-for-hyper-v>

- Hyper-V TPM config
<https://techcommunity.microsoft.com/blog/itopstalkblog/how-to-run-a-windows-11-vm-on-hyper-v/3713948>
- IIS Configuration – Configure application pool identity
<https://learn.microsoft.com/en-us/iis/manage/configuring-security/application-pool-identities>
- "key protector could not be unwrapped" Error
<https://learn.microsoft.com/en-us/answers/questions/2287161/server-2022-2025-hyper-v-new-instance-error-key-pr>
- Microsoft TPM/CPU requirements for Windows 11
<https://learn.microsoft.com/en-us/windows/whats-new/windows-11-requirements>
- PowerShell – DNS Zone Listing – Get-DnsServerZone
<https://learn.microsoft.com/en-us/powershell/module/dnsserver/get-dnsserverzone>
- PowerShell – DNS Zone Management – Add-DnsServerPrimaryZone
<https://learn.microsoft.com/en-us/powershell/module/dnsserver/add-dnsserverprimaryzone>
- PowerShell – Invoke-Sqlcmd Cmdlet – Invoke-Sqlcmd
<https://learn.microsoft.com/en-us/powershell/module/sqlserver/invoke-sqlcmd>
- PowerShell – SQL Server Cmdlets – SQLServer module overview
<https://learn.microsoft.com/en-us/powershell/module/sqlserver/>
- PowerShell – SQL Server Module Installation – Installing the SqlServer module
<https://learn.microsoft.com/en-us/sql/powershell/download-sql-server-ps-module>
- Project-Specific Documentation – README for CosmosDB Emulator Deployment
<https://rob-das-win.azurewebsites.net/html/README.html#development-database-azure-cosmos-db-emulator>
- System Properties (sysdm.cpl) – Accessing system settings
<https://learn.microsoft.com/en-us/windows/win32/shell/executing-control-panel-items>
- TFS Migration – Migrating from TFS to Azure DevOps Services
<https://learn.microsoft.com/en-us/azure/devops/migrate/migration-overview>
- Task Scheduler – Overview and usage
<https://learn.microsoft.com/en-us/windows/win32/taskschd/task-scheduler-start-page>
- Windows 10 to 11 upgrade <https://www.microsoft.com/en-ca/software-download/windows11>

- Windows 11 system requirements <https://www.microsoft.com/en-ca/windows/windows-11-specifications>
- Windows Server Manager – Overview and usage
<https://learn.microsoft.com/en-us/windows-server/administration/server-manager/server-manager>
- Windows Server Update Services (WSUS)
<https://learn.microsoft.com/en-us/windows/deployment/update/waas-manage-updates-wsus>
- WSUS and Configuration Manager
<https://learn.microsoft.com/en-us/troubleshoot/mem/configmgr/update-management/wsus-maintenance-guide>